

IBM Data Engineering Course | Coursera

NoSQL Database

Summary

Module one

Key-Value NoSQL Databases

Basics of NoSQL

Lesson 1: Overview of NoSQL

Lesson Objectives

- Define the term **NoSQL**
 - Describe **NoSQL technologies**
 - Understand the **history and evolution** of NoSQL databases
 - List **4 key reasons** for using NoSQL databases
-

What is NoSQL?

NoSQL = "Not Only SQL" – not "No SQL"

- Refers to a **family of non-relational** databases
 - These databases are **not based** on traditional relational DBMS (i.e., tables, rows, columns)
 - Also called **non-relational databases**
 - Common trait: **Flexible, schema-less, scalable**, and designed for distributed architecture
-

Why Use NoSQL?

Feature	Description
 Flexible Schema	Schema-less models allow for unstructured/semi-structured data
 Scalable Architecture	Easy horizontal scaling as demand grows
 Developer Productivity	Developers can model data closer to application needs

Feature	Description
🌟 Fault Tolerance & HA	Native support for high availability and fault tolerance in distributed systems

🕒 History of NoSQL

Era	Highlights
1970–2000	RDBMS dominance: Oracle, MySQL, IBM DB2, SQL Server
Apollo Era	IBM IMS (hierarchical DB) used in space missions (non-relational)
2000s	Dot-com boom created need for high-scale, high-availability DBs
White Papers	📄 Google's MapReduce for large-scale processing

📄 Amazon's ****Dynamo**** for distributed key-value store

| **Late 2000s** | Rise of open-source NoSQL DBs: MongoDB, Cassandra, CouchDB, Redis, Neo4j |

| **2010s+** | Cloud-based DBaaS models emerged: IBM Cloudant, Amazon DynamoDB |

🧠 Common NoSQL Types in Social Media Example

Database Type	Use Case in Social Media	Example DB
Document	User profiles	MongoDB
Column	Activity feeds	Apache Cassandra
Key-Value	User sessions, caching	Redis
Graph	Friends & relationships	Neo4j

🌱 Summary

- **NoSQL** = Not Only SQL → a family of non-relational databases.
 - Designed for modern, high-scale applications with **distributed**, **flexible**, and **fault-tolerant** architectures.
 - **No fixed schema** means agility and adaptability.
 - Emerged due to the scalability limits of RDBMS during the web boom.
 - NoSQL includes several models (Document, Key-Value, Column, Graph), each optimized for specific use cases.
 - Widely used today in **DBaaS** formats like IBM Cloudant & Amazon DynamoDB.
-

💡 Key Terms

NoSQL: Non-relational database technologies designed for scale and flexibility

DBaaS: Database-as-a-Service

Horizontal Scaling: Adding more machines to distribute load

Fault Tolerance: Ability to continue operation despite hardware failure

✖ Lesson 2: Characteristics of NoSQL Databases

🎯 Lesson Objectives

- Identify the **main characteristics** of NoSQL databases
- Understand the **4 types** of NoSQL databases
- Explain the **benefits** of adopting NoSQL
- Recognize NoSQL's **open-source roots and design patterns**

🔑 Key Characteristics of NoSQL

Trait	Description
🧱 Non-relational	Does not use traditional table-based structure (RDBMS)
🌐 Distributed by Design	Built for scalability across clusters, racks, or data centers
🔑 Unique Keys & Sharding	Uses global unique keys to support partitioning (sharding)
📦 Flexible Schemas	No fixed structure—schemas can evolve over time
✂ Specialized for Use Cases	Each type fits a particular pattern better than general-purpose RDBMS
🔧 Developer-Friendly	Easier and faster data modeling & iteration

🧱 Four Types of NoSQL Databases

Type	Description	Example Use Case	Tools / Tech
Key-Value	Data stored as key-value pairs	Caching, sessions	Redis, Amazon DynamoDB
Document	Stores JSON-like documents	User profiles, CMS	MongoDB, CouchDB
Column-Family	Column-based storage, optimized for reads	Event logs, analytics	Apache Cassandra, HBase

Type	Description	Example Use Case	Tools / Tech
Graph	Nodes & edges for relationships	Social networks, recommendations	Neo4j, ArangoDB

⚠ Note: These categories **can overlap**, and definitions aren't always strict.

Open Source Origins

- Many NoSQL tools **originated in the open-source community**
- Commercial support often coexists with open-source:
 - IBM Cloudant (built on CouchDB)
 - DataStax (supports Apache Cassandra)
 - MongoDB Inc. (commercial MongoDB with open-source core)

Why NoSQL? — Benefits Breakdown

Benefit	Description
 Scalability	Easily scale out horizontally to meet changing demands
 Performance	High concurrency + large datasets = fast response times
 High Availability	Clusters with redundancy ensure minimal downtime
 Cloud Ready	Designed for distributed cloud infrastructure (low cost vs. mainframes)
 Cost Effective	Cut database costs while maintaining performance
 Flexible Schema	No locking or downtime needed to update schema
 Intuitive Structures	JSON, key-value, and graph formats map naturally to app structures
 Special Capabilities	Geospatial queries, replication, HTTP APIs, etc.

Why *Not* Always NoSQL?

- While NoSQL is powerful, **RDBMS still wins** in:
 - Complex transactions requiring full ACID compliance
 - Systems needing **strong consistency**, referential integrity
 - Applications with fixed, well-structured data models
-

✓ Summary

- NoSQL = non-relational, horizontally scalable, schema-flexible DBs
 - There are **four main types**: Key-Value, Document, Column, Graph
 - NoSQL databases emerged from **open-source innovation**
 - Benefits include scalability, high availability, cloud-readiness, flexibility, and cost savings
 - NoSQL databases are better for **modern app needs**, while RDBMS still holds ground in **specific transactional use cases**
-

📚 Key Vocabulary

Sharding: Splitting database into chunks across servers

Flexible Schema: Ability to change/add fields on the fly

High Availability: Always accessible, even in case of failure

DBaaS: Database-as-a-Service (fully managed)

Open Source: Software with publicly accessible source code

📘 Lesson 3: NoSQL Database Types & Use Cases

🎯 Lesson Objectives

- Describe the characteristics of **key NoSQL types**
 - Identify **real-world use cases** for each type
 - List common **vendors** in the NoSQL ecosystem
 - Understand how **MongoDB works in CMS scenarios**
-

🌱 NoSQL Database Types Overview

Type	Format	Key Benefit	Best For
📄 Document Store	JSON / BSON	Flexible schema & hierarchical data	CMS, e-commerce, catalogs
💬 Key-Value Store	{key: value}	Ultra-fast read/write	Caching, sessions, gaming
📊 Column-Family	Column groups	Time-series & analytical queries	IoT, personalization, analytics
🕸 Graph Store	Nodes + Edges	Relationship-driven queries	Social networks, recommendations
🗄 Wide-Column Store	Table + columns	Hybrid SQL-like but flexible schema	Big data, enterprise content

Document Store Databases

Characteristics:

- Schema-less structure (JSON/BSON)
- Documents can vary in structure within same collection
- Good for hierarchical or nested data
- Supports CRUD efficiently
- Horizontally scalable via **sharding**

Use Cases:

-  **Content Management Systems (CMS)** — storing articles, users, media (e.g., *MongoDB*)
-  **E-Commerce Catalogs** — dynamic product attributes (e.g., *Couchbase*, *Amazon DocumentDB*)

Vendors:

- MongoDB
 - Couchbase
 - Amazon DocumentDB
-

Key-Value Stores

Characteristics:

- Simple: key → value
- Super fast lookups
- Ideal for distributed systems & session caching
- Excellent horizontal scalability

Use Cases:

- ⚡ Web caching (e.g., *Redis*, *Memcached*)
- 🎮 Session storage in games, apps (e.g., *DynamoDB*)

Vendors:

- Redis, Memcached, Amazon DynamoDB
-

Column-Family Stores (a.k.a Columnar DBs)

Characteristics:

- Stores data in **columns** not rows
- Efficient for **queries on specific attributes**
- Scales well with large, sparse data

Use Cases:

-  IoT sensors and time-series data (e.g., *Apache Cassandra*)
-  User behavior and personalization (e.g., *HBase*)

Vendors:

- Apache Cassandra, HBase
-

Graph Databases

Characteristics:

- Data modeled as **nodes**, **edges**, and **properties**
- Prioritizes **relationships**
- Efficient traversal queries

Use Cases:

-  Social networks (e.g., *Neo4j*)
-  Recommendation engines (e.g., *Amazon Neptune*)

Vendors:

- Neo4j, Amazon Neptune, ArangoDB
-

Wide-Column Stores

Characteristics:

- Similar to RDBMS but **with flexible schemas**
- Stores data in column families
- Built for **horizontal scaling** and **fault tolerance**

Use Cases:

-  Real-time big data analytics (e.g., *Apache HBase*)
-  Enterprise content like employee/inventory records (e.g., *Cassandra*)

Vendors:

- Apache HBase, Apache Cassandra
-

MongoDB in Content Management Systems (CMS)

Content Structure in MongoDB

MongoDB stores each content item as a document:

```
{
  "_id": 1,
  "title": "Sample Blog Post",
  "content": "This is the content of the blog post...",
  "author": {
    "name": "John Doe",
    "email": "john@example.com"
  },
  "tags": ["mongodb", "blogging"],
  "comments": [
    {
      "text": "Great post!",
      "author": "Emily Johnson"
    }
  ]
}
```

- Each blog post = 1 document
 - Tags, authors, comments are **subdocuments/arrays**
 - Supports **evolving schemas** and nested content
-

Indexing & Search

MongoDB allows **text-based indexing** for search:

```
db.articles.createIndex({ subject: "text" });

db.posts.find({ $text: { $search: "digital life" } });
```

- Allows searching inside string-based fields
- Uses **stemming** to match word roots

Scaling MongoDB for CMS

MongoDB supports **horizontal scaling** using **sharding**:

Scenario:

- Company has 100M customers → scales to 200M
- Vertical scaling = expensive
- Horizontal scaling via sharding = linear growth

➡ Sharding = Split content across servers for load balancing

➡ Zone-based sharding = Global distribution for geo-performance

Recap

Topic	Key Takeaways
NoSQL Types	Document, Key-Value, Column, Graph, Wide-Column
Flexibility	Schema-less & adaptable to fast-changing data needs
MongoDB in CMS	JSON document model for flexible content structures
Performance & Scaling	MongoDB scales horizontally and supports text indexing
Real-World Fit	NoSQL = ideal for modern apps, personalization, and global content delivery

Key Commands

```
// Create a text index
db.articles.createIndex({ subject: "text" })

// Search using text query
db.posts.find({ $text: { $search: "digital life" } })
```

Lesson 4: Key-Value NoSQL Databases

Lesson Objectives

- Understand the **architecture of Key-Value databases**
 - Identify their **strengths and limitations**
 - Describe **use cases** suitable for this type
 - List **popular implementations** in the market
-

🌱 Refresher: NoSQL Categories

Type	Key Feature	Best For
🧠 Key-Value	Simple key-to-value mapping	Session data, caching, shopping carts
📄 Document	JSON-like document storage	CMS, product catalogs, user content
📊 Column-Family	Column-grouped data	IoT, time-series analytics
🕸 Graph	Nodes + edges for relationships	Social networks, recommendations

🔑 Key-Value Database Architecture

```
Structure = { "key1": value1, "key2": value2, ..., "keyN": valueN }
```

- Internally resembles a **hash map**
- The **value is a blob** (opaque to the database engine)
- **No schema**, no structure enforcement
- Supports fast **CRUD** (Create, Read, Update, Delete)

Key Features:

Feature	Description
✅ Simplicity	Least complex of all NoSQL types
⚡ High performance	Optimized for speed over structure
🌐 Horizontal scaling	Data sharded easily across many nodes using key ranges
🔒 Atomic per key	Single-key transactions are atomic
🧱 Opaque values	Data inside values cannot be queried unless parsed at the application level

📦 Use Cases for Key-Value Databases

Use Case	Why It Fits
🔑 Session management	Unique key per session; fast read/write
👤 User profile storage	Lightweight, unconnected data
🛒 Shopping carts	One key → one cart → fast updates with minimal querying
🎮 In-app settings	Key for each user or feature, stored and fetched quickly

🚫 When NOT to Use Key-Value Stores

- ❌ When data has **many-to-many relationships** (e.g., friends in social networks)
- ❌ When you need **multi-key transactions** with ACID guarantees
- ❌ When you want to **query based on value** fields
- ❌ When your app requires **rich indexing or filters**

In such cases, consider **Document** or **Graph** databases instead.

🔧 Examples of Key-Value Databases

Tool	Notes
Amazon DynamoDB	Highly scalable, managed NoSQL service by AWS
Redis	In-memory data store, great for caching
Oracle NoSQL	Part of Oracle's cloud-native stack
Aerospike	High-performance, real-time analytics
Riak KV	Distributed and eventually consistent store
MemcachedB	Caching-focused, memory-efficient
Project Voldemort	Developed by LinkedIn for distributed key-value storage

✅ Summary

- **Key-Value databases** are the simplest NoSQL model: { key → value }
 - Optimized for **speed, scalability, and simplicity**
 - Excellent for **session storage, in-app settings, and non-relational isolated records**
 - Limited for **complex querying, relationships, and value-based search**
 - Examples: **Redis, DynamoDB, Aerospike, Memcached**, etc.
-

📁 Example (Pseudo-code)

```
# Imagine storing session info in a key-value store like Redis

# Create
SET session:12345 '{"user_id': 'u001', 'expires': '2025-07-01'}"

# Read
GET session:12345

# Update
```

```
SET session:12345 '{"user_id': 'u001', 'expires': '2025-07-15'}"

# Delete
DEL session:12345
```

Lesson 5: Document-Based NoSQL Databases

Lesson Objectives

- Understand the **architecture and behavior** of document databases
 - Explain the **advantages** of flexible document schemas
 - Identify **ideal use cases** for document databases
 - Know when **not** to use this model
 - Recognize popular **document database platforms**
-

What is a Document Database?

- Builds on the **key-value model** but with **queryable values**
 - Each record is a **self-contained document** (usually JSON or XML)
 - Supports **flexible schema** — no need for all documents to have the same structure
 - Allows **indexing, querying**, and even analytical processing (e.g. via MapReduce)
 - Transactions are **atomic only at the single-document level**
 - **Horizontally scalable** through **sharding** based on a document key
-

Document Database Architecture

```
// Example Document (Blog Post)
{
  "_id": "post_001",
  "title": "Hello NoSQL World!",
  "author": {
    "name": "Yousef",
    "email": "yousef@example.com"
  },
  "tags": ["nosql", "mongodb", "databases"],
  "created_at": "2025-06-20T12:00:00Z",
  "comments": [
    {"text": "Great article!", "author": "Reader1"},
    {"text": "Thanks for sharing!", "author": "Reader2"}
  ]
}
```

- Everything is **nested and intuitive** — no joins or multiple tables
- Subdocuments and arrays are **first-class citizens**

✓ Use Cases

Use Case	Description	Example
 Event Logging	Each log entry is a document capturing all event metadata	Log aggregation services
 Online Blogging	Users, posts, comments → each as separate but connected documents	CMS, blogs, forums
 Mobile/Web App Backend	Store operational and session data like metadata, user settings, state	Social apps, chat apps

🚫 When NOT to Use Document Databases

Scenario	Recommended Alternative
 Multi-document ACID transactions	Use relational DBs (e.g., PostgreSQL)
 Data naturally fits normalized/tabular structure	Use RDBMS
 Heavy joins or deep relationship queries	Consider Graph databases

 If you **find yourself forcing data into aggregates**, it might be time to rethink your schema design.

🌍 Popular Document-Based NoSQL Databases

Platform	Description
MongoDB	Most widely adopted, native JSON/BSON
IBM Cloudant	CouchDB-compatible, DBaaS solution
Couchbase	Built for mobile sync & fast queries
Apache CouchDB	RESTful JSON interface, great for sync
RavenDB	.NET-friendly document store
OrientDB	Combines document + graph model
Terrastore	Focused on dynamic clusters

Querying in Document DBs

```
// MongoDB-style find
db.posts.find({
  tags: "nosql",
  "author.name": "Yousef"
})
```

- You can query **nested fields**, **arrays**, and even do **text search** (with indexes).
- Powerful yet intuitive for developers who think in **objects**, not tables.

✓ Summary

Feature	Value
Format	JSON or XML
Schema	Flexible, self-describing documents
Scalability	Horizontal (via sharding)
Indexing & Querying	Yes, on document content and subfields
Transaction Model	Atomic per document only
Ideal For	CMS, blog platforms, event logs, RESTful API backends
Not Ideal For	Complex transactions, deep normalization, highly relational datasets
Popular Platforms	MongoDB, CouchDB, Cloudant, RavenDB, Couchbase, OrientDB

🔗 Pro Tip

Think of document stores as **mini-JSON-based databases** inside a big NoSQL engine — perfect for today's unstructured, fast-moving digital world. If your data talks REST and walks JSON, it probably belongs here. 😊

🧱 Lesson 6: Column-Based NoSQL Databases

🎯 Lesson Objectives

By the end of this lesson, you will be able to:

- Explain the **architecture** of column-based NoSQL databases
 - Describe their **technical strengths and weaknesses**
 - Identify **ideal use cases**
 - Name **leading technologies** in this category
-

Column-Based Architecture: Bigtable-Style

Column-based NoSQL databases were inspired by **Google's Bigtable** and are sometimes called:

- **Columnar databases**
- **Wide-column stores**
- **Bigtable clones**

Structure Overview

Component	Description
Row	Identified by a unique key
Column	Contains data for a specific attribute
Column Family	A logical group of columns accessed together
Sparse Layout	Rows may have different columns — no need for uniform schema

Column Family: UserBehavior

```
-----
RowKey: user_123
- clicks: 5
- purchases: 2
- last_login: 2025-06-20
RowKey: user_456
- clicks: 13
- purchases: 0
```

Technical Features

Feature	Description
 Column-Oriented Storage	Optimized for analytical queries — read only the columns you need
 Sparse Data Support	Columns can exist in some rows and not others
 Efficient Compression	Similar column types → better compression and storage efficiency
 Horizontal Scalability	Sharded across clusters like other NoSQL types
 Atomicity	Only row-level atomicity , not cross-row or full ACID
 TTL Support	Built-in time-to-live for auto-expiring data
 Counters	Built-in support for increment operations

Use Cases

Use Case	Why It Works
 Scientific Research	Works well with large structured datasets from experiments
 Business Intelligence (BI)	OLAP & reporting benefit from column-specific access
 Data Warehousing	Stores structured but massive data across departments
 Financial Analytics	Supports statistical aggregation & time-series analysis
 IoT Data Streams	Efficient real-time storage for sensor & telemetry data
 Enterprise Logging	Event logs stored per app with rowKey + timestamp layout
 E-commerce Analytics	Querying purchases, views, inventory in high-volume systems

Example: IoT Logging with Trucks

RowKey: truck_4380_20250620_13:00

- speed: 88
- location: "N31.42 E36.01"
- engine_status: "normal"
- dashcam: "image1234.jpg"

- Each truck logs a **row per timestamp**
 - Only relevant columns are stored (sparse)
 - Easy aggregation: fuel usage, driver performance, etc.
-

When to Avoid Column-Based Databases

Scenario	Why It's a Problem
 Multi-row ACID transactions	No built-in support for full transactional safety
 Frequently changing schemas	Column families are rigid in query tuning stages
 Highly relational data	Not optimized for complex joins
 Early development prototyping	Schema patterns may evolve too quickly

🔥 Popular Column-Based Databases

Technology	Description
Apache Cassandra	Highly scalable, distributed wide-column store
Apache HBase	Hadoop-compatible, strong for OLAP + big data
Hypertable	Performance-optimized Bigtable-style DB
Accumulo	Based on Bigtable, secure + fine-grained access control

🏠 Column vs. Row-Oriented for Analytics

Feature	Row-Oriented (RDBMS)	Column-Based (NoSQL)
Storage	Row-by-row	Column-by-column
Speed for OLAP	Slower	🚀 Faster for select queries
Schema flexibility	Strict schema	Medium flexibility
Ideal for	OLTP (transactions)	OLAP (analytics)

✅ Summary

Key Takeaways

Column-based NoSQL = inspired by Bigtable → **optimized for analytics**

Uses **column families** grouped by access pattern

Best for **sparse, large, structured datasets**

Horizontal scalability + **high compression** = lower storage + faster access

Popular for BI, IoT, logging, data warehousing, and time-series use cases

🔗 Query Optimization Tip

Instead of querying everything like:

```
SELECT * FROM orders;
```

You can query just what you need:

```
SELECT total_price FROM orders WHERE product_id = 'P101';
```

That's where column-based databases shine ✨

❄️ Lesson 7: Graph-Based NoSQL Databases

🎯 Lesson Objectives

- Describe the **architecture** of graph databases
 - Understand their **core strengths and weaknesses**
 - Identify **ideal business and technical use cases**
 - Name top vendors offering graph-based NoSQL solutions
-

🧬 What is a Graph Database?

A **Graph NoSQL Database** is based on graph theory — where data is represented as:

- 🧑 **Nodes** (entities like users, locations, products)
- 🔗 **Edges** (relationships like *friend-of*, *bought-with*, *located-near*)

These relationships are **first-class citizens**, just as important as the data itself.

🧠 Simple Example:

```
[User: Yusuf] --[LIKES]--> [Product: Noise-Cancelling Headphones]
[User: Yusuf] --[FRIENDS_WITH]--> [User: Ahmad]
[Product] --[SAME_CATEGORY_AS]--> [Another Product]
```

⚙️ Key Features

Feature	Description
🏠 Graph Model	Nodes and edges with attributes
🧪 ACID Compliant	Supports transactional consistency (unlike most other NoSQL types)
🕒 Efficient Traversal	Quick exploration of relationships (e.g. friends-of-friends)
🔄 Bi-directional Queries	Relationships can be navigated in both directions
🚫 Limited Horizontal Scale	Sharding is difficult due to interconnected nature

✅ Ideal Use Cases

Use Case	Why Graph DB Fits
 Social Networks	Represent users, friends, likes, posts, comments — deeply interconnected
 Recommendation Engines	Suggest items based on user-product-user relationships
 Maps & Routing	Model streets, routes, intersections for shortest path queries
 Network Analysis	Analyze internet, telecom, or organizational structures
 Knowledge Graphs	Connect structured + unstructured data (think: Google search graph!)

❌ Not Ideal For

Scenario	Reason
 Heavy Horizontal Scaling Needed	Sharding a graph breaks traversal paths, hurting performance
 Bulk Updates on Node Sets	Not optimized for updating attributes across large node sets
 Massive Tabular Data Processing	Better suited to Column Stores or RDBMS

🚀 Popular Graph NoSQL Databases

Database	Highlights
Neo4j	Most popular; uses Cypher query language
Amazon Neptune	Fully managed AWS graph DB (SPARQL, Gremlin support)
OrientDB	Combines graph + document DB features
ArangoDB	Multi-model: graph, document, key-value
JanusGraph	Scalable graph DB built over Cassandra/HBase
Apache Giraph	Optimized for large-scale graph processing (Hadoop)

🔍 Example: Friend-of-Friend Query

```
// Neo4j example
MATCH (me:Person {name: "Yousef"})-[:FRIEND_OF]->(friend)-
[:FRIEND_OF]->(fof)
RETURN DISTINCT fof.name
```

- Find all **friends of friends** for user “Yousef”
- One simple query — lightning fast

Summary

Characteristic	Graph NoSQL DBs
Data Model	Graph: Nodes + Edges
Best For	Social networks, recommendations, routing
Strengths	ACID support, fast traversal, strong relationship focus
Limitations	Horizontal scaling, mass updates
Top Tools	Neo4j, Amazon Neptune, OrientDB, ArangoDB

Visual Summary

Entity: [User]
Relationship: --LIKES--> or --FRIEND_OF-->
Traversal: FAST
Querying: Cypher / Gremlin / SPARQL
Scaling: Limited horizontally
Consistency: ACID compliant 

Wrap-Up: NoSQL Database Categories Quick Recap

Type	Structure	Best For	Example DBs
Key-Value	Key + Value	Caching, sessions	Redis, DynamoDB
Document	JSON/BSON docs	CMS, blogs, web/mobile apps	MongoDB, Couchbase
Column	Column families	Analytics, BI, IoT, warehousing	Cassandra, HBase
Graph	Nodes + Edges	Social graphs, routing, recommendations	Neo4j, Neptune, ArangoDB

Lesson 8: NoSQL Database Deployment Options

Lesson Objectives

By the end of this lesson, you will be able to:

- Identify common **NoSQL database deployment methods**
 - List **examples and vendors** for each deployment option
 - Explain the **benefits and challenges** of each strategy
-

1. On-Premises Deployment

Description:

Database hosted entirely within the organization's own physical infrastructure or private data center.

Example:

Bank uses Oracle NoSQL on its own secured servers.

Advantages:

-  **Full control** over hardware, OS, configurations, and security
-  **Regulatory compliance** (GDPR, HIPAA, etc.)

Challenges:

-  **High upfront costs** (hardware + IT staff)
 -  **Limited scalability** during demand spikes
-

2. Cloud Deployment

Description:

Hosted by public cloud providers like AWS, GCP, or Azure, with web-based access.

Example:

Startup deploys MongoDB on AWS EC2 or Amazon DocumentDB.

Advantages:

-  **Scalability:** Elastic resource scaling on demand
-  **Cost-effective:** Pay-as-you-go pricing
-  **Automation:** Managed backups, patches, monitoring

Challenges:

-  **Internet dependency** for access
 -  **Security concerns** (shared environment)
-

3. Hybrid Deployment

Description:

Mix of on-premises and cloud-based components.

📌 Example:

A fintech app stores client data in on-prem DB for compliance, while analytics workloads are run in AWS Lambda.

✅ Advantages:

- ⚖️ **Flexibility:** Sensitive data on-prem, dynamic workloads in cloud
- 💾 **Disaster recovery:** Cloud used as off-site backup

❌ Challenges:

- 🔗 **Integration complexity** between local & remote
 - 👤 **Higher admin overhead**
-

🔧 4. Database-as-a-Service (DBaaS)

📄 Description:

Fully managed database services where the vendor handles infrastructure, security, patching, and more.

📌 Example:

Dev team uses Azure Cosmos DB or Firebase for a mobile app backend.

✅ Advantages:

- 🚀 **Quick deployment:** Ready in minutes
- 💻 **Developer focus:** No DBA needed

❌ Challenges:

- ⚙️ **Limited infrastructure control**
 - 🕵️ **Security risk:** Sensitive data in 3rd-party hands
-

📦 5. Containerized Deployment

📄 Description:

Databases are packaged in Docker containers for consistent deployment across different environments.

📌 Example:

Using Docker Compose to deploy MongoDB in dev, staging, and prod.

✅ Advantages:

- 🚚 **Portability:** Runs the same everywhere
- 🍃 **Lightweight:** Less resource-hungry than VMs

❌ Challenges:

- 🧠 **Learning curve:** Requires knowledge of Docker & orchestration (Kubernetes)
 - 🛠️ **Orchestration complexity:** Scaling, networking, recovery setup
-

⚡ 6. Serverless Deployment

📖 Description:

Databases scale and provision automatically, abstracting away server infrastructure completely.

📌 Example:

Mobile app stores data in **Google Cloud Firestore** or **Amazon DynamoDB (serverless mode)**.

✅ Advantages:

- 🧑‍🔧 **Auto-scaling:** Handles sudden spikes
- 📄 **Usage-based billing:** Pay only for read/write ops

❌ Challenges:

- 🚫 **Limited visibility/control**
 - 🧱 **Not ideal for all workloads:** Complex joins, custom storage needs
-

🔄 Recap Table: Deployment Options

Type	Pros	Cons	Example Vendors
🏢 On-Premises	Full control, compliance	Costly, hard to scale	Oracle, IBM, MongoDB Enterprise
☁️ Cloud	Scalable, cost-effective, automated	Connectivity and security issues	AWS, GCP, Azure
🌐 Hybrid	Flexible, resilient	Integration and management complexity	AWS + Local PostgreSQL

Type	Pros	Cons	Example Vendors
 DBaaS	Low-maintenance, fast dev cycles	Less control, security concerns	Firebase, MongoDB Atlas, Azure SQL
 Containerized	Portable, efficient	Requires orchestration tools, learning curve	Docker, Kubernetes, MongoDB, Redis
 Serverless	Auto-scaled, cost-saving	Not suitable for all use cases	DynamoDB (serverless), Firestore

🔥 Visual Deployment Spectrum

```
[On-Premises] <-- more control | less scalable
[Hybrid]      <-- balance
[Cloud / DBaaS] <-- less control | more scalable
[Containerized / Serverless] <-- automation + flexibility
```

🌟 Final Takeaway

Choosing the right NoSQL deployment model is like choosing the right car for a road trip:

- **On-premises:** You buy the car 🚗
- **Cloud:** You rent it when needed 🚗
- **DBaaS:** You get a driver too 🚗
- **Containerized:** You drive your own modular, detachable car 🚗
- **Serverless:** You teleport to your destination 🚀

Lesson's Summary

- NoSQL means Not only SQL.
- NoSQL databases have their roots in the open-source community.
- NoSQL database implementations are technically different from each other.
- Adopting NoSQL databases has several benefits, including storing and retrieving session information and event logging for apps.
- The four main categories of NoSQL databases are Key-Value, Document, Wide Column, and Graph.
- Key-value NoSQL databases are the least complex architecturally.
- Document-based NoSQL databases use documents to make values visible for queries.

- In document-based NoSQL databases, each piece of data is considered a document, which is typically stored in either JSON or XML format.
- Column-based databases spawned from the architecture of Google's Bigtable storage system.
- The primary use cases for column-based NoSQL databases are event logging and blogs, counters, and data with expiration values.
- Graph databases store information in entities (nodes) and relationships (edges).

Glossary – Basics of NoSQL

Term	Definition
ACID	An acronym for Atomicity, Consistency, Isolation, and Durability , ensuring reliable processing of database transactions in traditional relational databases.
Atomic	An indivisible operation in a transaction that either fully completes or is rolled back entirely, preserving consistency.
BASE	An alternative to ACID: Basically Available, Soft state, Eventually consistent — prioritizes availability and scalability over strict consistency.
Bigtable	A NoSQL system developed by Google , designed for managing massive data with high performance and scalability.
Caching	Temporary storage of frequently accessed data in high-speed memory , improving system response time.
Cluster	A group of connected servers or nodes working together to manage data with high availability and fault tolerance.
Column database	A NoSQL model storing data in columns (column families) instead of rows, ideal for large-scale, analytical workloads. Examples: Apache Cassandra, HBase .
CRUD	Acronym for Create, Read, Update, Delete , representing the core operations on data.
DBaaS	Stands for Database as a Service , a managed cloud service that handles hosting, scaling, and maintenance of databases.
Document	A NoSQL model that stores data in semi-structured documents like JSON or BSON , grouped into collections.
Graph database	A NoSQL model designed for highly connected data, using nodes (entities) and edges (relationships). Examples: Neo4j, OrientDB .
Horizontal scaling	Adding more nodes or machines to increase capacity and performance, commonly done using sharding .
Indexing	Creating special data structures to improve the speed of data retrieval based on key fields.

Term	Definition
JSON	JavaScript Object Notation , a lightweight, human-readable data format widely used in NoSQL databases.
Key-value	A NoSQL model where data is stored as unique keys associated with value blobs , suitable for simple, high-speed access.
Normalized	A database design approach that reduces redundancy and maintains integrity by organizing data into related tables.
NoSQL	Stands for " Not Only SQL ", referring to database systems that provide flexible, non-tabular data models beyond traditional relational databases.
Sharding	Partitioning a database into smaller pieces called shards , each managed independently, enabling horizontal scaling.
TTL	Stands for " Time to Live ", defining how long a piece of data remains in the system before automatic deletion.
XML	Extensible Markup Language , a structured, human-readable data format used in NoSQL and other systems.

Working with Distributed Data

Lesson 1: ACID versus BASE Operations

Learning Objectives:

- Define the ACID and BASE acronyms.
- Explain the key differences between ACID and BASE.
- Identify common use cases for each consistency model.

ACID vs. BASE Comparison Table

Feature	ACID	BASE
Acronym	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventually consistent
Database Type	Relational Databases (e.g., MySQL, Oracle)	NoSQL Databases (e.g., MongoDB, Cassandra)
Data Consistency	Strong, immediate consistency	Eventual consistency
Availability	Lower availability in favor of consistency	High availability, prioritizes system uptime
Durability	Guaranteed—data persists even after power or network failure	Durability is not always immediate
Transactions	Reliable, consistent multi-step operations	May sacrifice consistency for responsiveness

Feature	ACID	BASE
Scalability	Vertical scaling	Horizontal scaling (more scalable)
Best Use Cases	Financial systems, banking, OLTP (online transaction processing)	Social networks, e-commerce profiles, media streaming platforms
Examples	Oracle, PostgreSQL, MySQL	MongoDB, Cassandra, Couchbase, Amazon DynamoDB

What is ACID?

- **Atomicity:** All steps in a transaction succeed, or none do.
- **Consistency:** Data remains accurate and valid after the transaction.
- **Isolation:** Transactions do not interfere with each other.
- **Durability:** Once a transaction is committed, the data remains even if there's a system failure.

 **ACID is ideal** for applications where correctness and accuracy are critical, such as **banking and payment systems**.

What is BASE?

- **Basically Available:** The system guarantees availability, even in case of partial failures.
- **Soft State:** Data may change over time due to replication delays or eventual updates.
- **Eventually Consistent:** System will become consistent at some point, but not immediately.

 **BASE is great** for systems where **availability and scalability** matter more than immediate consistency, such as **Netflix, Uber, Spotify, and social media platforms**.

Key Takeaway

ACID focuses on **data integrity and correctness** — best for high-value transactions. **BASE** emphasizes **availability and scalability** — best for high-volume, user-driven applications.

Both models are valid; choosing one depends on your **business needs and technical context**.

Lesson 2: Distributed Databases

Learning Objectives

- Understand what distributed databases are
- Define **fragmentation** and **replication**
- Identify the **advantages** and **challenges** of distributed systems

Key Concepts of Distributed Databases

Concept	Description
Distributed Database	A collection of interconnected databases physically located in different sites, communicating via a network. Typically follows the BASE model.
Fragmentation / Partitioning / Sharding	Breaking large data sets into smaller pieces across multiple nodes. Improves manageability and performance. Done using a key.
Replication	Creating redundant copies of data fragments on two or more nodes to ensure availability in case of node failure.
BASE Model	Focuses on availability over consistency . Data might be temporarily inconsistent but will become consistent eventually.

Fragmentation Techniques

Method	Description	Example
Lexical Grouping	Keys are grouped alphabetically or by range	Keys starting with A–C go to Server A
Key-based Grouping	Group all records with the same key together	All transactions from store ID 143 on Server B

Replication: Advantages and Disadvantages

Advantages	Disadvantages
Increases availability across nodes	Requires synchronization to avoid inconsistencies
Provides data redundancy	Updates at one node must replicate to all others
Improves fault tolerance	May cause latency or conflict if not managed properly

✅ Advantages of Distributed Databases

- **High Availability:** Data is available even if one server/node fails
 - **Performance Gains:** Querying is faster due to local data access
 - **Scalability:** Easy to add new servers to the cluster
 - **Global Reach:** Suitable for modern, internet-scale applications
 - **Continuous Operation:** No reliance on a central server
-

⚠️ Challenges of Distributed Databases

Challenge	Explanation
Concurrency Control	Multiple users modifying data in different locations might cause conflicts or inconsistencies
Data Synchronization	Changes must propagate to all replicas to maintain coherence
Limited Transactions	Full ACID support is rare; transactions are often limited to single fragments or nodes
Consistency Trade-offs	Developers must balance between availability and consistency via tunable settings

🔗 Developer Control Options

- Choose how many nodes must respond for a write/read to be considered successful
 - Control consistency levels per operation
 - Use **write-to-one/read-from-many** or **write-to-all/read-quorum** strategies
-

📌 Key Takeaway

Distributed databases offer **scalability**, **availability**, and **global reach**, but they trade off strict consistency and full transactional support.

They use **fragmentation** to split data and **replication** to prevent data loss — operating under the **BASE model** for eventual consistency.

🌱 MongoDB Design Patterns – Summary Table

Pattern	Purpose	Use Case Example	Key Benefits
Approximation	Estimates values when exact precision is not required	Tracking Instagram followers for celebrities	Reduces database writes and maintains statistically valid approximations
Attribute	Represents documents with various characteristics or optional fields	E-commerce product listings with different sets of attributes	Simplifies queries and minimizes indexing requirements
Polymorphic	Stores different types of related documents in the same collection	Logging customer communication across email, SMS, chat	Easy implementation; allows querying across one unified collection
Outlier	Handles documents that don't fit the general data pattern	Celebrity with millions of followers in a dataset dominated by small accounts	Isolates atypical data to avoid performance issues or bloated collections
Bucket	Groups and summarizes time-series or streaming data	IoT sensor data for weather reporting (e.g., average temperature per hour)	Reduces document volume, improves indexing, enables pre-aggregated analytics

🧠 Key Takeaway

MongoDB design patterns are **best practices** that optimize data modeling for performance, scalability, and maintainability. Choosing the right pattern depends on the **data access pattern**, **query frequency**, and **application-specific needs**.

📖 Lesson 3: CAP Theorem

🎯 Learning Objectives

By the end of this lesson, you will be able to:

- Define the **CAP Theorem**.
 - Describe the characteristics of **Consistency**, **Availability**, and **Partition Tolerance**.
 - Understand how **CAP** impacts the design and trade-offs of NoSQL databases.
-

What is CAP Theorem?

The **CAP Theorem** (also known as **Brewer's Theorem**) states that a distributed database system can **only guarantee two out of the following three** properties at the same time:

1. **Consistency (C):**
Every node returns the most recent write or an error.
2. **Availability (A):**
Every request receives a response, even if it may not contain the latest data.
3. **Partition Tolerance (P):**
The system continues to function despite network failures or communication breaks between nodes.

Since **network partitions are inevitable** in distributed systems, Partition Tolerance becomes a baseline requirement. This forces systems to choose between:

- **CP (Consistent + Partition Tolerant):** e.g., **MongoDB**
- **AP (Available + Partition Tolerant):** e.g., **Apache Cassandra**

CAP in Action – NoSQL Examples

Database	CAP Focus	Behavior
MongoDB	CP	Prioritizes consistency over availability during a partition.
Cassandra	AP	Ensures availability; consistency is eventual and tunable.

Why It Matters

- CAP helps explain **why distributed systems can't be perfect**.
- It guides **architectural decisions** for modern scalable applications.
- Choosing between **C** or **A** depends on the **application's business needs**:
 - Use **CP** when data accuracy is critical (e.g., financial systems).
 - Use **AP** when uptime is essential (e.g., global content services).

Key Takeaways

- CAP theorem defines the **trade-offs in distributed database systems**.
 - **Partition Tolerance is mandatory** in NoSQL systems.
 - NoSQL systems prioritize either **Availability** or **Consistency**, but not both.
 - CAP classification helps determine the right database for the right use case.
-

Lesson 4: Challenges in Migrating from RDBMS to NoSQL

Learning Objectives

By the end of this lesson, you will be able to:

- Identify the best use cases for **Relational (RDBMS)** vs. **NoSQL** databases.
 - Understand the **key differences** between relational and NoSQL paradigms.
 - Recognize the **challenges and considerations** when migrating from RDBMS to NoSQL.
-

Relational vs. NoSQL: Not a Competition

- NoSQL is **not a direct replacement** for relational databases.
 - Both **RDBMS and NoSQL serve different needs** and often **coexist** in modern architectures.
 - Migration should be driven by **business and technical requirements**, not trends.
-

When to Use Each Database Type

Use RDBMS When...	Use NoSQL When...
Full data consistency is required	High performance and scalability are key
Data is structured and requires joins	Data is unstructured or semi-structured
You need multi-document transactions	You need flexible schemas or denormalized models
Complex relationships exist between data entities	You need availability over consistency (CAP Theorem)

Hybrid Solutions

In some scenarios, combining **RDBMS + NoSQL** makes sense.

Example: A scalable NoSQL store for user sessions + RDBMS for billing and transaction logic.

⚠️ Key Challenges in Migration

1. **Mindset Shift in Schema Design**
 - RDBMS: Schema is **data-driven** → design starts from entities and relationships.
 - NoSQL: Schema is **query-driven** → design starts from how your application queries data.
 2. **Normalization vs. Denormalization**
 - **RDBMS** favors normalized, relational tables.
 - **NoSQL** often **denormalizes data** to optimize query speed and performance.
 - Data might be duplicated across collections to serve different query needs.
 3. **Trade-off Between Consistency and Availability**
 - Many **NoSQL systems prioritize availability** over strict consistency (see **CAP Theorem**).
 - For real-time systems (e.g., social media, global apps), **availability is more critical**.
 4. **Limited Support for Advanced Operations**
 - NoSQL databases **may not support**:
 - Multi-document transactions (in all cases)
 - Complex joins
 - Deep aggregations
 - You must plan around these **limitations** during migration.
-

🧠 Key Takeaways

- **RDBMS and NoSQL each solve different problems**—choose based on your use case.
 - Successful migration requires **understanding NoSQL principles**, especially around schema, scalability, and consistency.
 - **Start with the queries**: In NoSQL, the way you access data determines how you store it.
 - **Hybrid architectures** combining both RDBMS and NoSQL are common and effective.
-

📖 Lesson 5: Data Modeling in Relational and Document-based Databases

🎯 Learning Objectives

- Compare how the same data is modeled in relational vs. document databases.
- Understand the complexities of querying data in each type.
- Explain the implications of data duplication.
- Describe schema evolution considerations in both models.

Relational vs. Document-Based Data Modeling

Relational Database	Document-Based Database (NoSQL)
Organizes data in tables with fixed schemas	Organizes data in flexible documents (e.g., JSON)
Uses relationships and foreign keys	Embeds related data within self-contained documents
Avoids data duplication via normalization	Allows denormalization to improve read performance
Example: Separate tables for <code>Books</code> , <code>Authors</code> , <code>Genres</code> , <code>BookGenres</code>	Example: A single document stores book info including author and genres

Example: Library Catalog Data

- **Relational Model:** Data is split into multiple normalized tables with foreign keys.
 - **Document Model:**
 - {
 - "`_id`": 1,
 - "`title`": "The Great Gatsby",
 - "`author`": "F. Scott Fitzgerald",
 - "`isbn`": "978-0743273565",
 - "`published_year`": 1925,
 - "`genres`": ["Fiction", "Classic"]
 - }
-

Complexities of Querying

Relational Query (SQL)	Document Query (MongoDB)
Requires multiple JOINS and GROUP BY clauses	Queries are simpler and flatter
Complex query for getting books, authors, genres	Example: <code>db.books.find({}, {title: 1, author: 1, genres: 1})</code>
Increases complexity and processing time	No joins needed, as all relevant data is embedded

Data Duplication and Denormalization

- **Relational DB:** Update author name once in `Authors` table.
- **Document DB:** Update author name in **every document** where it appears.
 -  *Trade-off:* Better read performance.
 -  *Challenge:* Risk of inconsistency and larger storage size.

Schema Evolution Considerations

Relational Database

Schema is rigid and **must be altered with SQL commands**

Example: `ALTER TABLE books
ADD new_column data_type`

May require **data migration scripts**

Document Database (MongoDB)

Schema is flexible — you can **add/remove fields anytime**

Example: `db.books.update({ _id: 1 }, {
$set: { newField: "value" } })`

Changes apply to specific documents, no migration needed

Key Takeaways

- **Relational databases** favor structure, consistency, and normalization.
 - **Document databases** prioritize flexibility, performance, and simplicity.
 - Query complexity is higher in relational models due to joins and grouping.
 - Document-based queries are simpler but may require **duplicated data**.
 - Schema changes are **easier and less disruptive** in document databases.
-

Lesson 6: Vector Databases

Learning Objectives

By the end of this lesson, you will be able to:

- Define what a vector database is.
 - Understand how vectors represent data like text, images, and audio.
 - Describe the benefits of vector databases.
 - Recognize popular vector database tools.
-

What Is a Vector Database?

A **vector database** is designed to store, manage, and index massive quantities of **high-dimensional vector data** efficiently.

— *IBM*

These databases are becoming increasingly important due to the rise of **Large Language Models (LLMs)** such as **OpenAI's GPT**.

What Are Vectors?

- Vectors are **numerical representations** of data (e.g., text, images, audio).
- Generated using **embedding functions**, often based on ML models or feature extraction.
- Example:
 - **"Dog"** → [2.1, -0.3, 7.2, 9.6, 6.1]
 - **"K9"** → Same vector as "Dog"
 - **"Animal"** → [1.9, -0.4, 7.2, 8.0, 6.3] (close but not identical)

Each number in a vector is called a **dimension**. Vectors may have **dozens to thousands** of dimensions.

✅ **Key Insight:** The **closer** the vectors are in space, the **more similar** the items are in meaning or context.

Why Use Vector Databases?

Unlike traditional databases that rely on **exact matches**, vector databases enable:

- **Semantic search** — retrieve results based on meaning, not just literal terms.
- **Similarity search** — find related data based on vector "closeness."

Use Case Examples:

- Recommend movies or songs based on preferences.
- Suggest similar products while online shopping.
- Summarize or cluster user messages based on topic or tone.

🔍 Vectors that are mathematically closer are **faster to search** and **more relevant** in context.

Popular Vector Databases

Database	Key Features
Chroma	- Open-source- Stores embeddings + metadata- Supports search and document embedding
Pinecone	- Fully managed- Real-time ingestion- Low latency- High scalability
Weaviate	- Open-source- GraphQL API- Handles billions of objects- Real-time updates- Efficient similarity search

Key Takeaways

- Vector databases are purpose-built for **semantic search** and **LLM applications**.
 - Vectors allow for the transformation of various data types into **numeric space**.
 - Items with **similar meanings** are placed **closely** together in the vector space.
 - **Chroma**, **Pinecone**, and **Weaviate** are currently leading solutions in this space.
-

Lesson's Summary

- ACID stands for Atomicity, Consistency, Isolated, and Durable.
 - BASE stands for Basic Availability, Soft-state, and Eventual Consistency.
 - ACID and BASE are the consistency models used in relational and NoSQL databases.
 - Distributed databases are physically distributed across data sites by fragmenting and replicating the data.
 - Fragmentation enables an organization to store a large piece of data across all the servers of a distributed system by breaking the data into smaller pieces.
 - You can use the CAP Theorem to classify NoSQL databases.
 - Partition Tolerance is a basic feature of NoSQL databases.
 - NoSQL systems are not a de facto replacement for RDBMS.
 - RDBMS and NoSQL cater to different use cases, which means that your solution could use both RDBMS and NoSQL.
-

Glossary: Working with Distributed Data

Term	Definition
ACID	Acronym for <i>Atomicity, Consistency, Isolation, and Durability</i> — a set of properties that guarantee reliable processing of transactions in relational databases.
Atomic	An operation that is indivisible: it either completes fully or not at all, ensuring the database remains consistent.
Availability	In CAP, availability means the system stays operational and responsive, even during failures or network issues — a core trait of distributed systems.

Term	Definition
BASE	Stands for <i>Basically Available, Soft state, Eventually consistent</i> . BASE trades off strict consistency for better performance and scalability.
Basically Available	A system design where the database continues to operate even when some components fail.
CAP	A theorem stating that in a network partition (P), a system can only guarantee either <i>Consistency (C)</i> or <i>Availability (A)</i> — but not both simultaneously.
Consistency (CAP)	Ensures all nodes in the system reflect the same data at the same time after an update.
Consistent	Ensures a transaction moves the database from one valid state to another, preserving data integrity.
Denormalized	A design approach (often used in NoSQL) that duplicates data to improve query speed and reduce complex joins.
Durable	Guarantees that once a transaction is committed, the data will persist even in case of failures.
Eventually Consistent	The system becomes consistent over time as updates propagate to all nodes, assuming no new updates occur.
Isolated	Transactions occur independently — one transaction doesn't interfere with another.
Joins	A relational database operation that combines data from multiple tables using shared keys or columns.
Normalized	A relational database practice that reduces redundancy by structuring data into related tables.
Partition Tolerance	The system can still function despite network partitions or communication breakdowns — an essential part of distributed systems.
Replication	Copying data across multiple nodes to improve fault tolerance, availability, and read performance.
Sharding	Dividing a large database into smaller, more manageable parts (shards) and distributing them across different servers for scalability.
Soft State	Accepts that the system's state might temporarily be inconsistent, especially due to delays or concurrency, but expects eventual resolution.
Transactions	A sequence of database operations treated as a single logical unit — either all operations succeed, or none do.

Basics of MongoDB

Lesson 1: Overview of MongoDB

Objectives

After completing this lesson, you will be able to:

- Explain what MongoDB is
- List the core components of MongoDB
- Describe when and why to use MongoDB

Key Concepts

Term	Explanation
MongoDB	A NoSQL document database that stores data as JSON-like documents , not tables.
Document	A single record (like a Python dictionary or JSON object), e.g., a student profile.
Collection	A group of related documents (like a table in SQL), e.g., a <code>students</code> collection.
Database	A container for collections. E.g., <code>CampusManagementDB</code> holds <code>students</code> , <code>staff</code> , etc.
Fields	Key-value pairs inside documents (e.g., <code>firstName: "John"</code>). Fields are flexible and schema-less.
Subdocuments	Documents within documents, for grouping related data together.
Arrays	Lists of values (strings, numbers, even other documents) supported natively.
Data Types	MongoDB supports rich types like strings, dates (<code>ISODate</code>), numbers, etc.
High Availability	Achieved by data replication across nodes.
Scalability	MongoDB supports vertical scaling (bigger servers) and horizontal scaling (sharding).

Document Structure Example

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "studentID": 12345
}
```

Advantages of MongoDB

- Schema is flexible – add new fields anytime without altering tables
 - Ideal for **structured + unstructured** data
 - **Easy to model** data based on how it is accessed (read/write driven design)
 - Supports **subdocuments** and **arrays**
 - Offers **replication** and **horizontal/vertical scaling**
 - Works on-premise or with managed services (e.g., **MongoDB Atlas, IBM Cloud**)
-

Real-World Example

In a campus system:

- `students` and `staff` would be stored as **documents**
 - Grouped into **collections** like `studentsCollection` and `staffCollection`
 - Zip codes for the US can be added dynamically, showing MongoDB's **schema flexibility**
-

Summary

- MongoDB is a NoSQL, document-based DB optimized for flexibility and scale.
 - Unlike relational databases, there's **no need to define a rigid schema** upfront.
 - Documents are grouped into collections inside a database.
 - MongoDB handles a wide range of data types and supports **dynamic schema evolution**.
 - It's suitable for rapidly evolving apps needing performance, availability, and flexible data structures.
-

Lesson 2: Advantages of MongoDB

Objectives

By the end of this lesson, you will be able to:

- Identify the key advantages of MongoDB
 - Explain why MongoDB is well-suited for evolving and unstructured data
-

🔧 Key Benefits of MongoDB

Benefit	Description
1. Flexible Schema	MongoDB allows documents in the same collection to have different fields. Useful when data formats vary (e.g., UK vs. US addresses). This eliminates empty fields or rigid structures.
2. Schema Evolution	MongoDB supports dynamic schema changes. New fields can be added to documents on the fly (e.g., adding <code>contactless_delivery: true</code> to courier deliveries during the 2020 pandemic).
3. Code-First Approach	You can start coding and storing data immediately without first creating a rigid schema or complex table design like in RDBMS. No need for third-party ORMs.
4. Handling Unstructured Data	MongoDB can ingest and store data of various formats and shapes from different sources in one collection (e.g., stock market data from multiple vendors).
5. Querying & Analytics (MQL)	MongoDB Query Language (MQL) supports complex queries. Use aggregation pipelines for advanced analytics like grouping, filtering, and sorting large datasets.
6. High Availability	MongoDB offers built-in replication with replica sets (typically 3 nodes: 1 primary, 2 secondaries). Ensures fault tolerance and zero-downtime upgrades or failover.

📌 Examples

- **Courier System:** Added new field `delivery_contactless: true` during schema evolution without downtime or migration.
 - **Stock Aggregator:** Collected data from different APIs with varying schemas into a single MongoDB collection.
-

💡 Summary

- MongoDB provides a **schema-less, developer-friendly** environment.
 - Its **code-first** approach allows faster development and iteration.
 - Flexible document structure supports real-world data that often changes.
 - **Aggregation pipelines** offer powerful data analytics capabilities.
 - High availability is built-in via **replica sets**, enabling robust and fault-tolerant deployments.
-

Lesson 3: Use Cases for MongoDB

Objectives

- List key use cases for MongoDB
 - Understand how MongoDB supports industries like IoT, E-commerce, Finance, Gaming, and Real-Time Analytics
-

Core Use Cases of MongoDB

Use Case	Description
1. Many Sources – One View	MongoDB ingests data from different formats and sources (structured, unstructured, semi-structured) and unifies them using its flexible schema . This eliminates data silos and offers a consolidated view.
2. IoT (Internet of Things)	Billions of connected devices produce continuous, high-volume data (e.g., temperature readings every minute). MongoDB handles this via scalability and distributed storage , enabling efficient ingestion, querying, and analytics.
3. E-Commerce	Products have heterogeneous attributes (e.g., phones vs. books). MongoDB's document-based structure , with sub-documents and lists, is ideal for modeling these dynamic product schemas. It supports quick reads and easy updates.
4. Real-Time Analytics	MongoDB reduces reliance on complex ETL pipelines by enabling in-place querying and aggregation . This means real-time decision-making based on live data streams—even with semi- or unstructured data.
5. Gaming	Multiplayer games demand global data access and low latency. MongoDB's sharding (horizontal scaling) supports high throughput and quick access. Its flexible schema adapts easily to evolving game data models.
6. Finance	MongoDB supports high-speed operations (thousands/sec), encryption at rest and in transit , and field-level security . It's suitable for banks and trading platforms needing high availability , data protection , and real-time performance .

Security & Performance Notes

- **Encryption:** At rest, in transit, and even field-level
 - **Availability:** Supports high-reliability environments like banking with replica sets and failover
 - **Throughput:** Suitable for high-speed financial operations
-

Summary

MongoDB is a versatile NoSQL database designed to power modern, data-intensive applications across industries. Whether it's IoT, gaming, e-commerce, or finance, MongoDB delivers:

- A **schema-less structure** for diverse data
 - **Global scalability**
 - **Real-time analytics**
 - High **security** and **availability** standards
-

Lesson's Summary

- MongoDB is a document and NoSQL database.
 - MongoDB is easy to access by indexing.
 - Mongo DB supports various data types, including dates and numbers.
 - The database schema can be flexible when working with MongoDB.
 - You can change the database schema without involving complex data definition language statements.
 - Complex data analysis can be done on the server using Aggregation Pipelines.
 - The scalability MongoDB provides makes it easier to work across the globe.
 - MongoDB enables you to perform real-time analysis of your data.
-

Glossary: Basics of MongoDB

Term	Definition
Aggregation Pipeline	A framework in MongoDB for transforming and processing data through stages like filtering, grouping, sorting, and projecting.
Code-First	A development approach where application code defines the data model. MongoDB supports this via its flexible schema structure.
Collection	A group of related MongoDB documents, similar to a table in relational databases but schema-less and JSON-like.
Database	A logical container in MongoDB for one or more collections; used for isolating data sets.
Document	A semi-structured data unit (like JSON or BSON) in MongoDB that represents a record. Stored in collections.
ETL (Extract, Transform, Load)	A data integration process that extracts data from sources, transforms it for business use, and loads it into MongoDB.
Expressive Querying	The ability to write complex, flexible queries using MongoDB's rich query language and aggregation tools.
High Availability (HA)	MongoDB's ability to remain operational during hardware or system failures, typically through replication and automatic failover.
JSON (JavaScript Object Notation)	A lightweight data format used by MongoDB for storing and exchanging data; human-readable and machine-parsable.
MQL (MongoDB Query Language)	MongoDB's native query language used to perform read and write operations on documents.
NoSQL	A category of databases (like MongoDB) that store data in non-tabular forms—ideal for flexibility and scalability.
Operational Data	Live, active data that is frequently read from or written to by applications.
Unstructured Data	Data without a predefined schema. MongoDB excels at storing unstructured or semi-structured data formats.

Getting Started with MongoDB

Lesson 1: CRUD Operations in MongoDB Using Mongo Shell

Learning Objectives

- Connect to a MongoDB database using MongoShell.
- Perform basic **CRUD** operations: **Create, Read, Update, Delete**.

What is Mongo Shell?

- A command-line interactive JavaScript interface provided by MongoDB.
 - Used for both data operations and administrative tasks.
 - Supports variable declarations, function calls, and direct interaction with databases.
-

Connecting to MongoDB

1. Connect using a **connection string**.
 2. Use `show dbs` to list all databases.
 3. Select your database with `use CampusManagement`.
 4. List collections with `show collections`.
-

CRUD Operations

Create

- **Insert a single document:**
 - `db.students.insertOne({`
 - `firstName: "John",`
 - `lastName: "Doe",`
 - `email: "john.doe@example.com"`
 - `})`
 - MongoDB automatically adds an `_id` field if not provided.
 - **Insert multiple documents:**
 - `const student_list = [{...}, {...}]`
 - `db.students.insertMany(student_list)`
-

Read

- **Find the first document:**
 - `db.students.findOne()`
 - **Find a specific document:**
 - `db.students.findOne({ email: "john.doe@example.com" })`
 - **Find all students with last name "Doe":**
 - `db.students.find({ lastName: "Doe" })`
 - **Count documents with a specific condition:**
 - `db.students.count({ lastName: "Doe" })`
-

Update

- **Replace an entire document:**
 - ```
db.students.replaceOne(
 { lastName: "Doe" },
 {
 firstName: "John",
 lastName: "Doe",
 email: "john.doe@campus.edu",
 mode: "Online"
 }
)
```
  - **Update specific fields using \$set:**
  - ```
db.students.updateOne(  
  { lastName: "Doe" },  
  { $set: { mode: "Online", email: "john.doe@campus.edu" } }  
)
```
 - **Update all documents:**
 - ```
db.students.updateMany({}, { $set: { mode: "Online" } })
```
- 

## Delete

- **Delete one document:**
  - ```
db.students.deleteOne({ email: "john.doe@example.com" })
```
 - **Delete many documents:**
 - ```
db.students.deleteMany({ lastName: "Doe" })
```
- 

## Key Takeaways

- **MongoShell** allows interactive command-line access to MongoDB.
  - CRUD = Create, Read, Update, Delete.
  - Common MongoDB shell functions include:
    - `insertOne`, `insertMany`
    - `findOne`, `find`, `count`
    - `replaceOne`, `updateOne`, `updateMany`
    - `deleteOne`, `deleteMany`
  - MongoDB returns acknowledgments after operations, including affected count and inserted IDs.
-

## Lesson 2: Indexes in MongoDB

### Learning Objectives

- Explain why indexes are important.
  - Use indexes to perform efficient searches and sorts.
  - Understand how MongoDB stores indexes.
- 

### What Are Indexes?

- Indexes are **data structures** that help MongoDB **locate documents faster**—without scanning the entire collection.
- Think of indexes like the **table of contents or alphabetical listings** in a book or dictionary.

### Real-Life Examples:

- A **library** organizes books by section, subject, and author.
- A **telephone book** sorts names alphabetically.
- A **dictionary** uses alphabetical indexing.

These are all ways to **quickly access specific data** without checking every single entry.

---

### Why Use Indexes in Databases?

Without indexes, MongoDB performs a **collection scan**—checking every document.

For example:

- If you often search by `courseId`, you should create an index on that field:
- `db.course_enrollment.createIndex({ courseId: 1 })`
  - `1` means **ascending order**.

This helps MongoDB **jump straight to matching documents** without scanning all data.

---

### Compound Indexes

- A **compound index** is an index on **multiple fields**.
- Example:
- `db.course_enrollment.createIndex({ courseId: 1, studentId: 1 })`

- Now MongoDB can **filter by** `courseId` and **automatically sort by** `studentId`—because both are in the index.
- 

## How MongoDB Stores Indexes

- MongoDB stores indexes as a **balanced tree structure** (like a B-tree).
- Each **index entry** includes:
  - The **indexed field(s)**
  - The **location** of the document on disk

This structure allows **fast searches**, whether you're filtering by **equality** or a **range of values**.

---

## Performance Benefits

- Indexes greatly improve **query performance**, especially on large datasets.
  - If a query involves **sorting on an indexed field**, MongoDB **does not need to sort** the results again.
- 

## Key Takeaways

- Indexes make **data retrieval fast and efficient**.
  - Always create indexes for fields used in **frequent searches or sorts**.
  - **Compound indexes** are helpful when you query or sort on **multiple fields**.
  - MongoDB uses **balanced tree structures** to store and manage indexes.
- 

## Lesson: Aggregation Framework in MongoDB

### Learning Objectives

By the end of this lesson, you will be able to:

- Explain what the aggregation framework is.
  - Identify and describe common stages of the aggregation pipeline.
  - Recognize when and why to use aggregation in real-world scenarios.
-



## What Is the Aggregation Framework?

- The **Aggregation Framework** (also called the **aggregation pipeline**) is a way to process and analyze data in **multiple stages**.
- It's similar to a **data assembly line**: documents enter from one end, pass through multiple stages (e.g., filtering, grouping), and come out transformed.

---

## Common Stages in the Aggregation Pipeline

| Stage                  | Purpose                                                                             |
|------------------------|-------------------------------------------------------------------------------------|
| <code>\$match</code>   | Filters documents based on conditions (like WHERE in SQL).                          |
| <code>\$group</code>   | Groups documents by a key (e.g., <code>courseId</code> ) and performs aggregations. |
| <code>\$project</code> | Reshapes the document: includes/excludes fields.                                    |
| <code>\$sort</code>    | Sorts the documents (1 = ascending, -1 = descending).                               |
| <code>\$count</code>   | Counts documents and assigns the count to a field.                                  |
| <code>\$merge</code>   | Writes the pipeline output into another collection.                                 |

---

## Example: Calculate Average Scores per Course

To calculate the average student scores in 2020 grouped by course:

1. Use `$match` to select documents where `year = 2020`.
2. Use `$group` to group by `courseId` and calculate the average using `$avg`.

```
db.courseResults.aggregate([
 { $match: { year: 2020 } },
 { $group: {
 _id: "$courseId",
 averageScore: { $avg: "$score" }
 }}
])
```

 You can even store the output using `$merge`:

```
{ $merge: "averageScores" }
```

---

## Other Practical Examples

### `$project` Example

```
{ $project: { firstName: 1, lastName: 1, _id: 0 } }
```

Only returns first and last name; `_id` is excluded.

### **\$sort Example**

```
{ $sort: { lastName: -1 } } // Sort by last name descending
```

### **\$count Example**

```
{ $count: "totalStudents" }
```

Counts documents in the current pipeline stage.

---

## **Use Cases for Aggregation**

- **Education:** Track student performance by course/year.
  - **E-commerce:** Calculate total or average sales per country/product.
  - **Finance:** Summarize transaction data by customer or region.
  - **Reporting & Analytics:** Build dashboards and performance metrics directly in MongoDB.
- 

## **Key Takeaways**

- MongoDB's **Aggregation Framework** is a powerful tool for **data transformation and analytics**.
  - It operates in **stages**, allowing flexible and modular queries.
  - Use it when you need **grouping, filtering, reshaping, sorting, or exporting** data.
  - With `$merge`, you can even **store your results** into a new collection.
- 

## **Lesson 4: Replication and Sharding in MongoDB**

### **Learning Objectives**

By the end of this lesson, you will be able to:

- Define **replication** and explain its purpose.
  - Describe the **benefits** and **limitations** of replication.
  - Understand what **sharding** is and how it helps scale databases.
  - Explain **election** and **failover** in replica sets.
-

## Replication: Ensuring High Availability

### Definition:

Replication in MongoDB is the process of **duplicating data across multiple nodes** to provide **redundancy** and **fault tolerance**.

### How it works:

- A **replica set** typically consists of **three nodes**:
  - **Primary node**: Receives all write operations.
  - **Secondary nodes**: Replicate data from the primary.
- Changes made to the **primary** are logged in an **Oplog** (operations log).
- Secondaries **observe** the Oplog and **replicate** the latest changes using timestamps.

### Important Note:

Replication **does not protect against accidental deletions**.

If a document is deleted on the primary, it is also deleted on all secondaries.

To recover from human errors, **backups and restores** are necessary.

---

## ✂ Benefits of Replication

| Benefit            | Description                                                                   |
|--------------------|-------------------------------------------------------------------------------|
| High Availability  | Keeps the application running even if a node fails.                           |
| Redundancy         | Multiple copies of data are always maintained.                                |
| Maintenance Ready  | Nodes can be updated one at a time (rolling updates) with no downtime.        |
| Automatic Failover | If the primary fails, MongoDB <b>automatically elects</b> a new primary node. |

---

## Primary Node Election Process

MongoDB ensures **only one primary** exists through a **voting mechanism**.

- A new primary is elected when:
    - The current primary fails.
    - A new replica set is initialized.
    - A manual failover is triggered (e.g., for maintenance).
  - Nodes with **up-to-date data** and **low replication lag** are prioritized.
-

## Sharding: Scaling Horizontally

### ✓ Definition:

Sharding is MongoDB's method of **horizontal scaling** by **partitioning large collections** across multiple servers (called **shards**).

### How it works:

- Collections are **split into smaller parts**, each stored on a different shard.
- MongoDB automatically routes queries to the correct shard(s) based on the **shard key**.

### Benefits of Sharding

| Benefit                 | Description                                                      |
|-------------------------|------------------------------------------------------------------|
| Increased Storage       | Enables you to store more data than a single server can handle.  |
| Better Performance      | Queries hit only relevant shards, reducing workload.             |
| Geographic Distribution | Data can be distributed regionally (e.g., US data on US shards). |
| Unified View            | Developers see one database, even though data is distributed.    |

---

### Key Takeaways

- **Replication** provides **redundancy**, **high availability**, and **automatic failover**, but does **not replace backups**.
  - **Sharding** allows MongoDB to handle **large datasets** and **high throughput** by distributing data across multiple machines.
  - Together, replication and sharding make MongoDB a **highly scalable and resilient** database platform.
- 

## Lesson 5: Accessing MongoDB from Python

### Learning Objectives

By the end of this lesson, you will be able to:

- Explain what `MongoClient` is.
  - Perform basic **CRUD operations** on MongoDB using **Python**.
  - Understand when to use **insert**, **find**, **update**, and **delete** methods effectively.
-

## 🌱 1. Introduction to MongoClient

### ✅ Definition:

`MongoClient` is a class from the `pymongo` library — the **official MongoDB driver for Python** — used to connect and interact with a MongoDB database.

### 🔴 Setup:

```
from pymongo import MongoClient

uri = "mongodb://your-cluster-url"
client = MongoClient(uri)

db = client["CampusManagementDB"] # Access the database
students_collection = db["students"] # Access the collection
```

⚠️ Best practice: Create only **one** `MongoClient` instance per application.

---

## 🔗 2. CRUD Operations Using Python

### ✅ Create (Insert)

```
Insert a single document
students_collection.insert_one({"firstName": "John", "lastName":
"Doe", "email": "john.doe@example.com"})

Insert multiple documents
students_collection.insert_many([
 {"firstName": "Jane", "lastName": "Smith"},
 {"firstName": "Alice", "lastName": "Wang"}
])
```

---

### 🔍 Read (Query)

```
Find first document (natural order)
student = students_collection.find_one()

Find by email
student = students_collection.find_one({"email":
"john.doe@example.com"})

Find all students with last name "Doe"
cursor = students_collection.find({"lastName": "Doe"})

Count matching documents
count = students_collection.count_documents({"lastName": "Doe"})
```

❗ `find()` returns a **cursor** object. Use a loop or `list(cursor)` to extract documents.

---

## Update

### a. Replace entire document:

```
Replace student document with new structure
students_collection.replace_one(
 {"lastName": "Doe"},
 {"firstName": "John", "lastName": "Doe", "email":
"john@campus.edu", "mode": "online"}
)
```

### b. In-place update with \$set (Preferred):

```
Add or modify specific fields
students_collection.update_one(
 {"lastName": "Doe"},
 {"$set": {"mode": "online", "email": "john@campus.edu"}}
)
```

### c. Update multiple documents:

```
Update all students to "online" mode (e.g., during lockdown)
students_collection.update_many({}, {"$set": {"mode": "online"}})
```

---

## Delete

```
Delete one student
students_collection.delete_one({"email": "john@campus.edu"})

Delete all students with last name "Doe"
students_collection.delete_many({"lastName": "Doe"})
```

---

## 3. Summary

| Feature                  | Description                                             |
|--------------------------|---------------------------------------------------------|
| MongoClient              | Used to connect to MongoDB from Python using pymongo.   |
| insert_one / insert_many | Add single or multiple documents.                       |
| find, find_one           | Retrieve documents from a collection.                   |
| replace_one              | Replaces entire document.                               |
| update_one, update_many  | Modify fields in one or more documents.                 |
| delete_one, delete_many  | Remove documents from the collection based on a filter. |
| count_documents          | Efficiently count how many documents match a condition. |

---

## Lesson's Summary

- CRUD operations consist of Create, Read, Update, and Delete.
  - The Mongo shell is an interactive command line tool provided by MongoDB to interact with your databases.
  - Indexes are an organized reference used to locate data quickly.
  - MongoDB stores data being indexed on the index entry and the document's location on disk.
  - Using an aggregation framework, you can perform complex analysis on the data in MongoDB.
  - You can build your aggregation process in stages such as match, group, project, and sort.
  - Replication is the duplication of data and any changes made to the data.
  - Replication provides fault tolerance, redundancy, and high availability for your data.
  - For growing data sets, you can use sharding to scale horizontally.
  - MongoClient is a class that helps you interact with MongoDB.
- 

## Glossary: Getting Started with MongoDB

---

### Core Concepts

| Term               | Definition                                                                                                                      |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Mongo shell</b> | An interactive command-line interface for MongoDB. It uses JavaScript-like syntax and is great for administration and querying. |
| <b>MongoClient</b> | A Python (or language-specific) object used to connect to a MongoDB server, enabling database interactions.                     |
| <b>CRUD</b>        | Acronym for <b>Create, Read, Update, Delete</b> – the four basic operations for manipulating data in databases.                 |

---

## Querying & Analytics

| Term                        | Definition                                                                                                                                                        |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Aggregation pipeline</b> | A framework in MongoDB for transforming data through stages like <code>\$match</code> , <code>\$group</code> , <code>\$sort</code> , and <code>\$project</code> . |
| <b>Indexing</b>             | The process of creating structures that allow faster data retrieval based on specific fields.                                                                     |
| <b>B+ Tree</b>              | A balanced tree data structure commonly used for building indexes in databases.                                                                                   |
| <b>Idempotent changes</b>   | Operations that produce the same result even if performed multiple times – useful for consistent data updates.                                                    |

---

## Data Distribution & High Availability

| Term                   | Definition                                                                                          |
|------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Replication</b>     | Keeping copies of your data on multiple servers (nodes) to ensure availability and fault tolerance. |
| <b>Oplog</b>           | A special log that stores write operations on the primary node; it helps sync secondary nodes.      |
| <b>Primary node</b>    | The main node in a replica set that handles all write operations.                                   |
| <b>Secondary node</b>  | Nodes that replicate data from the primary and can be used for read operations.                     |
| <b>Election</b>        | The process of selecting a new primary node if the current one becomes unavailable.                 |
| <b>Replication lag</b> | A delay between changes on the primary node and their reflection on secondary nodes.                |

---

## Scaling Techniques

| Term                      | Definition                                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Sharding</b>           | Breaking a large collection into smaller chunks (shards) to distribute data across multiple machines. |
| <b>Horizontal scaling</b> | Adding more machines to handle increased data or load (scales out).                                   |
| <b>Vertical scaling</b>   | Upgrading an existing machine's resources like CPU or RAM (scales up).                                |

---



# MongoDB Cheat Sheet — Quick Reference

## Basic Connection

```
mongosh "URI"
mongosh --host mongodb0.example.com --port 28015
mongosh "mongodb://mongodb0.example.com:28015" --username alice --
authenticationDatabase admin
```

---

## Database & Collection

```
show dbs // List all databases
use <database_name> // Switch or create a database
db.createCollection("name") // Create a new collection
show collections // List collections in current DB
```

---

## Insert Documents

```
db.collection.insert({ name: "John", age: 22 }) // Single insert
db.collection.insertMany([
 { name: "Alice", age: 25 },
 { name: "Bob", age: 30 }
]) // Bulk insert
```

---

## Find & Query Documents

```
db.collection.find() // Get all
documents
db.collection.find({ field: value }) // Filter by field
db.collection.find({ field: { $gt: 20 } }) // Greater than
db.collection.find({ field: { $lt: 50, $gt: 20 } }) // Range query
db.collection.find({ f1: v1, f2: v2 }) // AND query
db.collection.find({ $or: [{ f1: v1 }, { f2: v2 }] }) // OR query
```

## Sorting

```
db.collection.find().sort({ name: 1 }) // Ascending
db.collection.find().sort({ name: -1 }) // Descending
```

---

## Update & Delete

```
db.collection.updateOne({ name: "John" }, { $set: { age: 23 } })
db.collection.updateMany({ age: { $gt: 30 } }, { $set: { active: true } })

db.collection.deleteOne({ name: "John" })
db.collection.deleteMany({ active: false })
```

---

## Aggregation Framework

```
db.collection.aggregate([
 { $match: { field: value } },
 { $group: { _id: "$field", total: { $sum: 1 } } }
])
```

---

## Indexes

```
db.collection.createIndex({ field: 1 }) //
Single-field
db.collection.createIndex({ field1: 1, field2: -1 }) //
Compound
db.collection.getIndexes() //
List indexes
```

---

## Data Import/Export

```
mongoexport --db myDB --collection users --out users.json
mongoimport --db myDB --collection users --file users.json
```

---

## Pro Tips

- `ObjectId()` is autogenerated if not provided.
  - `_id` field is unique per document.
  - Use `findOne()` if you need just one match.
  - Use `countDocuments()` to count documents matching a filter.
  - `mongosh` supports JS-like scripting (define variables, use loops).
- 

## Cassandra Basics

### Apache Cassandra – Quick Overview

#### Definition

Apache Cassandra is an **open-source, distributed, decentralized, fault-tolerant, highly available, elastically scalable** NoSQL database.  
It was **created at Facebook**, inspired by **Amazon's Dynamo** (distribution model) and **Google's Bigtable** (data model).

---

## Cassandra vs MongoDB

| Feature              | Cassandra                                    | MongoDB                                               |
|----------------------|----------------------------------------------|-------------------------------------------------------|
| Architecture         | Peer-to-peer (no primary node)               | Primary-secondary                                     |
| Consistency          | Tunable (favors availability in CAP)         | Stronger consistency by default                       |
| Best for             | Write-heavy, real-time ingestion             | Read-heavy, document queries, flexible schema         |
| Scalability          | Linear, across data centers                  | Scalable with sharding, but less seamless             |
| Joins & Aggregations | No joins, limited aggregation                | Supports aggregations via aggregation framework       |
| Use cases            | Logging, metrics, time-series, user sessions | E-commerce, content apps, flexible search & reporting |

---

## Key Capabilities of Apache Cassandra

- **Decentralized** architecture (no single point of failure)
  - **High availability** across geographic regions
  - **Tunable consistency** (choose between strong or eventual)
  - **Extremely fast write performance**
  - **Linear scalability** without downtime
  - **Multi-data center replication**
  - **CQL (Cassandra Query Language)** – similar to SQL
  - **Append-only design** ideal for time-series and event-driven systems
- 

## Best Usage Scenarios

Cassandra excels when:

- ✓ You have **write-heavy workloads**
  - ✓ Your data is **append-only** (like logs or sensor data)
  - ✓ You query using a **primary/partition key**
  - ✓ You need **geographical distribution**
  - ✓ You require **100% uptime** and **fault tolerance**
- 

## Common Use Cases

-  **IoT & Time Series Data:** Weather sensors, clickstreams
-  **User Profiles:** Session state, preferences (Netflix, Spotify)

-  **E-commerce analytics:** Transaction logs for real-time insights
  -  **Access Logs & Authentication Attempts**
  -  **Operational metrics storage**
  -  **Streaming platforms:** Logs, interactions, recommendations
- 

## **Limitations**

-  No joins or foreign keys
-  Limited built-in aggregation
-  Not ideal for transactional banking systems (e.g., account balances)

For complex analytics and joins → Pair Cassandra with **Apache Spark**.

---

## **Final Thought**

Cassandra trades **relational flexibility** for **massive scalability and high availability**.  
Use it when **speed, uptime, and distributed scale** matter more than complex querying.

---

## **Apache Cassandra Architecture – Cheat Sheet**

### **1. Overview**

Apache Cassandra is a **distributed, decentralized, and fault-tolerant** NoSQL database system built for:

- **High availability**
- **Linear scalability**
- **Massive write throughput**

No single point of failure. All nodes are **equal peers**.

---

## 🌱 2. Cassandra Topology

| Component                 | Description                                                 |
|---------------------------|-------------------------------------------------------------|
| Node                      | The basic unit. Holds data and responds to client requests. |
| Cluster                   | A set of connected nodes.                                   |
| Datacenter                | A grouping of nodes for replication control.                |
| Peer-to-Peer Architecture | No primary node. Each node can handle read/write requests.  |

---

## 💬 3. Internal Communication – Gossip Protocol

- Nodes share state info every second with up to 3 other nodes.
  - Helps track which nodes are **up/down**, **token ownership**, and **data consistency**.
  - Enables **eventual metadata consistency** across the cluster.
- 

## 🧠 4. Node-Level Components

| Component                 | Function                                                                         |
|---------------------------|----------------------------------------------------------------------------------|
| Memtable                  | In-memory write buffer (per table). Eventually flushed to disk as SSTables.      |
| Commit Log                | Append-only log to ensure <b>durability</b> . Data written here before Memtable. |
| SSTables                  | Immutable disk-based files created from Memtable flushes.                        |
| Bloom Filter              | Probabilistic structure to <b>avoid unnecessary disk reads</b> .                 |
| Partition Index & Summary | Helps map keys to their location in SSTables.                                    |

---

## 📝 5. Write Path (Simplified)

1. Client sends **write** to any node (acts as **coordinator**).
  2. Data is written to:
    - **Commit Log** (durability)
    - **Memtable** (fast access)
  3. When thresholds are met → Memtable is flushed to disk → becomes an **SSTable**
  4. SSTables are later **compacted** for performance.
-

## 6. Read Path

1. Coordinator receives **read request**
  2. Data is fetched from:
    - **Memtable**
    - **Bloom Filter**
    - **Cache**
    - **SSTables (via Index, Summary, Data.db)**
  3. Results are **merged and reconciled** from multiple sources (if needed).
- 

## 7. Data Distribution

| Mechanism          | Description                                                        |
|--------------------|--------------------------------------------------------------------|
| <b>Partitioner</b> | Hashes partition keys to tokens for <b>even distribution</b>       |
| <b>Token Ring</b>  | Each node manages a <b>range of tokens</b>                         |
| <b>Replication</b> | Data is duplicated across nodes based on <b>replication factor</b> |
| <b>Snitches</b>    | Understand <b>network topology</b> to optimize data placement      |

---

## 8. Data Model

- Based on **column-family** model
  - Data is stored in **tables**
  - Each **row** has a unique **partition key**
  - Columns can vary per row (schema flexible)
- 

## 9. Read & Write Optimization

- **Writes** are fast: Append-only and in-memory.
  - **Reads** optimized via caching, indexes, and bloom filters.
  - Supports **consistency levels** (e.g., ONE, QUORUM, ALL)
  - Uses **hinted handoff**, **read repair**, and **tunable consistency** for data accuracy.
- 

## 10. Maintenance Features

| Feature                     | Function                                                |
|-----------------------------|---------------------------------------------------------|
| <b>Compaction</b>           | Merges SSTables to reduce duplication and fragmentation |
| <b>Compression</b>          | Reduces disk footprint                                  |
| <b>Backup &amp; Restore</b> | Snapshot-based for DR                                   |

| Feature  | Function                                      |
|----------|-----------------------------------------------|
| Security | Auth, RBAC, encryption in transit and at rest |

---

## 11. Administrative Tools

- **nodetool**: Cluster management (repair, cleanup, etc.)
  - **CQL (Cassandra Query Language)**: SQL-like interface for developers
- 

## 12. Security Features

- Authentication & Role-based access
  - TLS encryption for client-node and node-node communication
  - Pluggable auditing and logging
- 

## 13. Summary

| Feature                | Description                                             |
|------------------------|---------------------------------------------------------|
| <b>Scalable</b>        | Add nodes with zero downtime                            |
| <b>Always-on</b>       | No single point of failure                              |
| <b>High throughput</b> | Especially write-heavy workloads                        |
| <b>Decentralized</b>   | Every node is equal                                     |
| <b>Use cases</b>       | IoT, analytics, logging, session data, time-series apps |

---

Here's a **concise, structured summary** of the **Key Features of Apache Cassandra**, tailored for quick reference or integration into a study guide or documentation.

---

## **Key Features of Apache Cassandra**

### 1. Distributed & Decentralized

| Feature                             | Description                                                                 |
|-------------------------------------|-----------------------------------------------------------------------------|
| <b>Distributed</b>                  | Data is spread across many nodes; appears as a single database to clients.  |
| <b>Decentralized (Peer-to-Peer)</b> | No master or primary node — all nodes are equal and can handle any request. |

| Feature         | Description                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| Gossip Protocol | Nodes sync state info every second with up to 3 others, maintaining metadata like availability, token range, etc. |

---

## 2. Data Distribution & Replication

- **Partitioning:** Data is grouped by **partition key**, hashed into **tokens**, and distributed across nodes.
  - **Token Ranges:** Each node is assigned specific token intervals and stores corresponding data.
  - **Replication:** Copies of data (replicas) are stored on multiple nodes.
    - **Replication Factor (RF)** controls how many replicas exist.
    - **Topology-Aware:** Aims to distribute replicas across racks and data centers.
  - **Write Path:** Parallel writes to all replica nodes with **no read-before-write**.
  - **Consistency Level:** Tunable per operation (e.g., ONE, QUORUM, ALL).
- 

## 3. Availability vs. Consistency (CAP Theorem)

| Concept             | Description                                                                 |
|---------------------|-----------------------------------------------------------------------------|
| Availability        | Always responsive, even during node failures.                               |
| Consistency         | Tunable by user — can opt for <b>strong</b> or <b>eventual</b> consistency. |
| Conflict Resolution | Happens <b>during reads</b> using timestamps and repair strategies.         |

---

## 4. Scalability

- **Horizontal Scaling:** Add or remove nodes **without downtime**.
  - **Linear Performance Growth:** More nodes = more throughput.
  - **Auto Load Balancing:** Existing nodes share token ranges with new ones.
- 

## 5. High Write Throughput

- **Parallelism:** Writes go to all relevant replicas concurrently.
  - **No Read-Before-Write:** In-memory writes to **Memtables**, logged in **Commit Logs**, flushed to **SSTables**.
  - **Write Efficiency:** Data appended to disk; compaction reconciles and optimizes later.
- 

## 6. Cassandra Query Language (CQL)



- SQL-like syntax: CREATE, INSERT, UPDATE, DELETE, SELECT.
- Designed for simplicity and **familiarity** to RDBMS developers.
- **Important Distinction:** Internals like joins, foreign keys, transactions work differently (or not at all).

---

## ✅ Summary

| Capability             | Description                                            |
|------------------------|--------------------------------------------------------|
| <b>Architecture</b>    | Distributed, decentralized, peer-to-peer               |
| <b>Availability</b>    | Prioritized over consistency; tunable per use case     |
| <b>Scalability</b>     | Add nodes seamlessly with linear performance gains     |
| <b>Fault Tolerance</b> | Built-in redundancy and graceful degradation           |
| <b>Performance</b>     | Especially optimized for <b>write-heavy workloads</b>  |
| <b>Interaction</b>     | CQL offers intuitive SQL-like access to a NoSQL system |

---

## Apache Cassandra - Key Features Summary

### 🧠 Learning Objectives

- ✓ Understand the distributed and decentralized nature of Cassandra.
- ✓ Explain Cassandra's replication strategy and consistency model.
- ✓ Describe how Cassandra scales horizontally.
- ✓ Identify Cassandra's high write throughput design.
- ✓ Interact with Cassandra using CQL (Cassandra Query Language).

## 🧱 1. Architecture: Distributed and Decentralized

### 📡 Distributed System

- ✓ Data is **distributed across multiple nodes**.
- ✓ Appears as a **unified database** to clients and applications.

### 🤝 Decentralized Design

- ✓ **No master/slave architecture** (unlike MongoDB).
- ✓ All nodes are **equal peers**.
- ✓ Any node can accept **read/write** requests.

### 📡 Gossip Protocol

- ✓ Nodes exchange state information every second.
- ✓ Maintains cluster health and metadata (node status, tokens, etc.).

# Each node gossips with 3 others per second  
 # Enables decentralized coordination

---

## 2. Data Distribution & Replication

### Partitioning Strategy

- Data is partitioned using a **partition key**.
- Partition key is hashed into **tokens**.
- Tokens are evenly distributed across the **token ring**.

### Replication

- **Replication Factor (RF)** determines how many replicas are stored.
- Replicas are stored **clockwise** in the ring.
- Distributed across **data centers and racks** (rack-aware).

# Example

Replication Factor = 3 # Three replicas per partition

---

## 3. Availability vs. Consistency

### CAP Theorem

- Cassandra favors **Availability** over **Consistency**.
- **Eventual consistency** by default.
- **Tunable consistency** per query: ONE, QUORUM, ALL.

### Conflict Resolution

- Resolved during **read** operations using **timestamps**.

# Read-time conflict resolution ensures eventual consistency

---

## 4. Elastic Scalability

### Horizontal Scaling

- Nodes can be **added or removed** with zero downtime.
- Cassandra automatically **rebalances token ranges**.

### Linear Performance Growth

- Throughput and storage increase **linearly** with node count.

# Add node -> Instant participation -> Load balancing kicks in

---

## 5. High Write Throughput

### Optimized for Writes

- No read-before-write operations.
- Writes go to:
  1. **Commit Log** (durability)
  2. **Memtable** (in-memory)
  3. Later flushed to **SSTables** on disk

### Compaction

- Merges SSTables over time for **efficient read** and space use.

# Append-only disk writes + Background compaction = High throughput

---

## 6. Cassandra Query Language (CQL)

### SQL-like Syntax

- Familiar syntax:
  - CREATE TABLE, INSERT, UPDATE, DELETE, SELECT

### Limitations vs SQL

- No joins
- Limited aggregation
- No foreign keys or referential integrity

```
-- CQL Example
CREATE TABLE users (
 id UUID PRIMARY KEY,
 name TEXT,
 email TEXT
);

SELECT * FROM users WHERE id = ?;
```

---

## Summary

| Feature          | Description                                       |
|------------------|---------------------------------------------------|
| Distributed      | Runs across multiple machines with unified access |
| Decentralized    | No single point of failure, peer-to-peer          |
| Scalable         | Add nodes easily, linear growth                   |
| Highly Available | Tunable consistency + fault tolerance             |

| Feature                                             | Description                    |
|-----------------------------------------------------|--------------------------------|
| Write-Optimized In-memory + append-only disk writes |                                |
| Query Language                                      | SQL-like CQL for easy adoption |

---

**Pro Tip** 💡 : Cassandra is ideal for **write-heavy, always-on, globally-distributed** apps with simple read patterns and known access keys (e.g. Netflix, Uber, Spotify, IoT, time-series logs).

---

## Apache Cassandra - Data Model (Part 1)

### Learning Objectives

- ✓ Describe the **logical entities** in Cassandra's data model.
- ✓ Explain the role of **primary keys** in Cassandra tables.
- ✓ Understand what **partition keys** are and how they work.
- ✓ Distinguish between **static** and **dynamic** tables.

## 1. Keyspaces and Tables

### Keyspace:

- A logical container for one or more tables.
- Defines global settings like the **replication strategy**.
- Recommended: Use one **keyspace per application**.

### Table:

- Logical structure for organizing data into rows and columns.
- Created using CQL (Cassandra Query Language).
- Can be created, dropped, or altered **without impacting active queries or updates**.

## 2. Primary Key Structure

### Primary Key:

- Mandatory when creating a table.
- Cannot be modified after creation.
- **Two main roles:**
  1. Optimizes query performance.
  2. Ensures row uniqueness.

### Components:

#### Partition Key (mandatory):

- Used to distribute data across nodes.
- Determines which node stores the data (via hashing).

#### Clustering Key (optional):

- Determines the order of rows within a partition.

### 3. Partitioning and Data Locality

- Data is grouped by partition key → each group is a partition.
- A hash function determines which node stores which partition.
- Example:  
A query like `SELECT \* FROM table WHERE groupid = 12` can be directed to one node only, improving performance in large clusters.

### 4. Table Types: Static vs. Dynamic

#### Static Table:

- Primary key includes only the partition key.
- Each partition has only one row.
- Example use case: Storing user profiles.

#### Dynamic Table:

- Primary key includes partition key + clustering key.
- Multiple rows per partition, ordered by the clustering key.
- Example use case: Storing users within groups, ordered by username.

### Summary

- ✓ Cassandra organizes data into **tables grouped in keyspaces**.
  - ✓ Primary keys define both data distribution (partition key) and row ordering (clustering key).
  - ✓ Static tables: Single row per partition.
  - ✓ Dynamic tables: Multiple rows per partition, ordered by clustering keys.
  - ✓ Proper primary key design is critical for performance and scalability.
- 

## Apache Cassandra – Data Model (Part 2)

### Learning Objectives

- ✓ Understand what a **clustering key** is.
- ✓ Describe the structure and behavior of **dynamic tables**.
- ✓ Apply **basic data modeling rules** for efficient query performance in Cassandra.

#### 1. Clustering Key

- ✓ The **clustering key** is the **second component** of a Cassandra primary key.
- ✓ It determines the **ordering of rows** within a partition (ascending by default).
- ✓ Can be **single** or **multi-column**.

Helps:

- ✓ Add **uniqueness** to rows within a partition.
- ✓ Improve **query performance** by organizing related data contiguously.

🧠 Example:

If the primary key is:

```
SQL: PRIMARY KEY ((groupid), age, username)
```

Then rows are stored:

- First **grouped by groupid** (partitioned).
  - Then **sorted by age**, and then by username within each age group.
- 

## 2. Dynamic Tables

- **Dynamic table:** A table whose primary key contains **both partition key and clustering key**.
  - Rows are added under the **same partition**, so the **partition grows dynamically**.
  - Efficient for use cases where:
    - You expect multiple related entries per partition.
    - Queries filter based on the clustering columns.
- 

## 3. Query Performance and Partition Optimization

**Why it's important:**

- Large partitions can have **hundreds of MBs** of data.
- If your query only needs a few KBs, it should **avoid scanning the entire partition**.

**Good Practice:**

Design clustering keys that **minimize the amount of data read**, by aligning clustering columns with query filters.

---

## 4. Data Modeling Guidelines

✓ **Choose a Partition Key That:**

- **Starts answering your query.**
- **Distributes data evenly** across the cluster.
  - e.g., Use `groupid` if there are many distinct groups of similar size.

## ✓ Choose a Clustering Key That:

- **Reduces scan range** within the partition.
  - **Orders rows in a way** that improves your most frequent queries.
- 

## 🔧 Example Use Case

Query:

```
SELECT * FROM groups WHERE groupid = 12 AND age = 32;
```

Suggested primary key:

```
PRIMARY KEY ((groupid), age, username)
```

This allows:

- Efficient partition lookup by `groupid`.
  - Fast in-partition scan by `age`.
- 

## ✓ Summary

- The **clustering key** defines the **row order** inside a partition and **boosts read performance**.
  - Use **dynamic tables** to allow flexible and scalable multi-row partitions.
  - A **well-designed primary key** reduces both inter-node and intra-partition reads.
  - **Start with your queries**, then design your data model to fit them.
- 

## 📘 Cassandra Query Language Shell (CQLSH) – Cheat Sheet

### 🎯 Learning Objectives

- ✓ Understand **what CQL is** and how it differs from SQL.
  - ✓ Know the **options for running CQL queries**.
  - ✓ Use the **CQL shell (cqlsh)** and its **command-line options**.
  - ✓ Apply **special CQLSH commands** such as `'CONSISTENCY'` and `'COPY'`.
-

## What is CQL?

- CQL (Cassandra Query Language): The main language for interacting with Cassandra clusters.
  - SQL-like syntax: Familiar but with key differences:
    -  No JOINS → denormalize and store pre-joined data.
    -  Insert/Update/Delete done in-memory without read-before-write.
  - Case-insensitive for keywords and identifiers.
    - `"Name"` (quoted) vs ``name`` (unquoted) → different behavior.
- 

## Ways to Run CQL Queries

1. **Programmatically:** Using drivers (Java, Python, Scala).
    - Default: Datastax Java Driver
  2. **Via CLI:** Using **CQLSH** (bundled with Cassandra).
- 

## CQLSH (Cassandra Query Language Shell)

- A **Python-based** CLI tool to interact with Cassandra.
- Connects to a **single node** in the cluster.

Common Usage:

```
cqlsh <hostname> --user <username> --password <password>
```

---

## CQLSH Command-Line Options

| Option                         | Description                                |
|--------------------------------|--------------------------------------------|
| <code>--help</code>            | Show help                                  |
| <code>--version</code>         | Show current version of <code>cqlsh</code> |
| <code>--user</code>            | Specify username                           |
| <code>--password</code>        | Specify password                           |
| <code>--keyspace</code>        | Connect to a specific keyspace             |
| <code>--file</code>            | Execute CQL commands from a file           |
| <code>--request-timeout</code> | Set query timeout (default: 10 sec)        |

---





## Key CQLSH Features

- Create / Alter / Drop:
    - KEYSPACES, TABLES
  - DML:
    - INSERT, UPDATE, DELETE, SELECT
  - Special commands:
    - CAPTURE, DESCRIBE, EXIT, PAGING, TRACING
- 



## Special Commands Explained



### CONSISTENCY

- **Tunable consistency** level for operations.
- Defines how many **replica nodes must respond** for success.

| Option          | Meaning                           |
|-----------------|-----------------------------------|
| ONE, TWO, THREE | Exact number of replicas          |
| QUORUM          | Majority of total replicas        |
| ALL             | All replicas                      |
| LOCAL_QUORUM    | Majority within local data center |



Example:

Cluster with 5 replicas (RF=5)

CONSISTENCY QUORUM → requires **3 successful responses**

---



### COPY

- Use for **small text-delimited datasets** (CSV)



#### Export:

```
COPY keyspace.table TO 'file.csv';
```



#### Import:

```
COPY keyspace.table FROM 'file.csv';
```



Note: Must match table structure and contain **primary key** values.

---



## Summary

- **CQL** is Cassandra's SQL-like query language, used for all data operations.
  - **CQLSH** is the official shell for interacting with Cassandra nodes.
  - Supports full CRUD operations and schema changes.
  - Special commands like `CONSISTENCY` and `COPY` enhance data control.
  - Always **design your schema based on queries**, and **write before read** is the default behavior.
- 

## Cassandra Glossary

- **CQL (Cassandra Query Language)**: SQL-like language used to interact with Cassandra.
  - **Keyspace**: Like a database in SQL; defines replication and groups tables.
  - **Table**: Stores data in rows and columns.
  - **Primary Key**: Uniquely identifies rows; made of:
    - **Partition Key**: Distributes data across nodes.
    - **Clustering Key**: Sorts data within a partition.
  - **Static Table**: Only has a partition key (one row per partition).
  - **Dynamic Table**: Has both partition and clustering keys (multiple rows per partition).
  - **Cluster**: A group of nodes storing data together.
  - **Peer-to-Peer**: All nodes are equal; no master/slave.
  - **Replication**: Copies of data stored across nodes for fault tolerance.
  - **Consistency**: Ensures data is the same across nodes (can be tuned).
  - **Availability**: System stays responsive even if some nodes fail.
  - **Scalability**: Easy to add nodes for more storage or traffic.
  - **Gossip Protocol**: Keeps nodes aware of each other's status.
  - **CAP Theorem**: Trade-off between Consistency, Availability, and Partition tolerance.
- 

## Lesson's summary

- Apache Cassandra is an open source, distributed, decentralized, elastically scalable, highly available, fault-tolerant, tunable, and consistent database.
- Apache Cassandra is best used by applications that require a database that is always available.
- Data distribution and replication take place in one or more data center clusters.
- Cassandra stores data in tables that are grouped in keyspaces.

- A clustering key specifies the order in which the data is arranged inside the partition (ascending or descending).
  - Dynamic table partitions grow dynamically with the number of entries.
  - CQL is the primary language for communicating with Apache Cassandra clusters.
  - CQL queries can be run programmatically using a licensed Cassandra client driver, or they can be run on the Python-based CQL shell client provided with Cassandra.
- 

## Working with Cassandra

### CQL Data Types – Summary Notebook

#### Objectives

By the end of this lesson, you should be able to:

- Identify the 3 main categories of data types in CQL.
  - Use appropriate data types when defining tables.
  - Understand how to model one-to-many and one-to-one relationships using collections and UDTs.
- 

### 1. Built-in Data Types

These are the standard, ready-to-use data types in Cassandra:

| Data Type                                | Description                                   |
|------------------------------------------|-----------------------------------------------|
| <code>ascii</code>                       | US-ASCII character string                     |
| <code>text</code> / <code>varchar</code> | UTF-8 encoded string                          |
| <code>boolean</code>                     | True/False values                             |
| <code>int</code>                         | 32-bit signed integer                         |
| <code>bigint</code>                      | 64-bit signed long                            |
| <code>float</code> / <code>double</code> | Decimal values                                |
| <code>blob</code>                        | Binary data (e.g., images, audio — up to 1MB) |

 Note: No complex JOINS in Cassandra. Store what you need, how you need it.

---

## 2. Collection Data Types

Used to store **grouped values** in a single column. Great for modeling **one-to-many** relationships.

### Collection Type

### When to Use

|                                    |                                                           |
|------------------------------------|-----------------------------------------------------------|
| <code>list&lt;type&gt;</code>      | Ordered collection, allows duplicates (e.g., job history) |
| <code>set&lt;type&gt;</code>       | Unordered, <b>unique</b> elements (e.g., email list)      |
| <code>map&lt;key, value&gt;</code> | Key-value pairs (e.g., {date: note} journal)              |

⚠️ Best for *bounded* collections. Avoid using collections for large, unbounded datasets (like chat logs or sensor streams).

---

## 3. User-Defined Types (UDTs)

Used to model **one-to-one** relationships by grouping multiple fields into a single column.

Example:

```
CREATE TYPE address (
 street text,
 city text,
 zip int
);

CREATE TABLE users (
 id UUID PRIMARY KEY,
 name text,
 home_address frozen<address>
);
```

💡 Think of UDTs like mini-objects stored inside a column.

---

## Example Use Case

```
-- Adding a list of jobs for a user
ALTER TABLE users ADD jobs list<text>;

-- Inserting jobs in order
UPDATE users SET jobs = ['Intern', 'Engineer'] WHERE id = 101;

-- Creating a UDT for location
CREATE TYPE location (
 lat double,
 lon double
);
```

---

## Key Takeaways

-  **Built-in types** are standard primitives (text, int, boolean, etc.)
-  **Collections** = embedded multi-values in one column (like arrays or dictionaries).
-  **UDTs** = grouped fields for logical, reusable column structures.
-  Avoid using collections for unbounded growth.
-  Use **list** for ordered, **set** for unique, and **map** for key-value pairs.

---

## Cassandra Keyspace Operations – Summary Notebook

### Objectives

- Understand what a keyspace is in Cassandra.
- Explain replication factor and strategy.
- Perform key operations: create, alter, and drop a keyspace.

---

### What is a Keyspace?

- A **keyspace** is the top-level container for tables in Cassandra.
- No default keyspace exists – you **must** create one before adding tables.
- Each table belongs to one keyspace.
- Think of it as Cassandra’s version of a “database” in SQL.

 Pro Tip: One keyspace per application is the recommended practice.

---

### Replication: Factor vs. Strategy

| Term                        | Definition                                                                            |
|-----------------------------|---------------------------------------------------------------------------------------|
| <b>Replication Factor</b>   | Number of copies of each piece of data across the cluster.                            |
| <b>Replication Strategy</b> | Defines how (and where) the replicas are distributed across nodes/racks/data centers. |

### Types of Strategies:

- `SimpleStrategy` → only for dev/testing.
- `NetworkTopologyStrategy` → **recommended for production**. Allows fine control over datacenter replication.

---

## Example: CREATE KEYSPACE

```
CREATE KEYSPACE intro_cassandra
WITH REPLICATION = {
 'class' : 'NetworkTopologyStrategy',
 'datacenter1' : 3,
 'datacenter2' : 2
};
```

- Total Replication Factor = 5
- 3 replicas in DC1, 2 in DC2

 **All replicas are equal** in Cassandra. No “primary” node drama here.

---

## How Replication Works

1. Data is hashed by the **partition key**.
2. Placed on a **primary node** (based on token ring).
3. Replicas are placed **clockwise**, considering **rack awareness**.
4. Data is duplicated across racks & data centers for fault tolerance.

## Visualization Example:

- 4 nodes in 1 datacenter, RF = 3
    - Original data → Node 1
    - Replica 1 → Node 3 (different rack)
    - Replica 2 → Node 4
- 

## Keyspace Operations

| Operation                 | Syntax Example                                             |
|---------------------------|------------------------------------------------------------|
| Create                    | CREATE KEYSPACE ...                                        |
| Modify (e.g. increase RF) | ALTER KEYSPACE intro_cassandra WITH REPLICATION = { ... }; |
| Delete                    | DROP KEYSPACE intro_cassandra;                             |

 Bonus: Cassandra **automatically snapshots** the keyspace before deletion – just in case you fat-fingered the `DROP`.

---

## Key Takeaways

-  Keyspaces organize tables and define how data is replicated.
  -  Replication **factor** = how many copies.
  -  Replication **strategy** = where to place them.
  -  Use `NetworkTopologyStrategy` in real-world deployments.
  -  Key operations: `CREATE`, `ALTER`, and `DROP`.
- 

## Cassandra Data Modeling & Querying – Best Practices Summary

### Purpose

Learn how to design efficient, scalable, and high-performance data models and queries in Apache Cassandra by aligning with its distributed architecture.

---

### Best Practices

#### 1. Improve Read and Write Performance

- Design models that allow data retrieval in a **single query**.
- Avoid joins by **pre-structuring** data to match query patterns.
- Enable **fast access** from distributed storage with minimal latency.

#### 2. Partition Correctly

- Choose **partition keys** that:
  - Avoid **hotspots** (don't overload a few nodes).
  - Do **not** have high cardinality (too many unique values).
  - Avoid **frequently updated** fields.
- Prevent **large partitions** (they degrade performance).

#### 3. Use Data Duplication

- Replicate data across multiple tables to support **diverse queries**.
- Optimize queries by **designing tables for specific query patterns**, even if this means some redundancy.

 In Cassandra, **denormalization is a feature**, not a bug.

#### 4. Avoid Full Table Scans

- Always **filter by partition key**.
- Avoid range scans without specifying a partition key.
- Scanning the full dataset is **expensive and slow** in large clusters.

## 5. Select Only Required Columns

- Use `SELECT column1, column2...` instead of `SELECT *`.
- Reduces **network bandwidth** usage and **query latency**.

## 6. Prefer Batch Updates

- Use `BEGIN BATCH ... APPLY BATCH` for grouped writes.
- Ensures **atomicity** and reduces **network overhead**.
- But: Batches are **not** for bulk loading unrelated rows.

---

## Summary of Key Guidelines

| Principle               | Benefit                                    |
|-------------------------|--------------------------------------------|
| Model based on queries  | Faster and simpler reads                   |
| Smart partitioning      | Even data distribution, better performance |
| Duplicate data          | Support multiple access patterns           |
| Avoid table scans       | Reduced load, better scalability           |
| Select specific columns | Faster queries, lower network usage        |
| Use batch writes wisely | Efficient and consistent multi-write ops   |

---

## Apache Cassandra Table Operations – Summary

### Objectives

After this lesson, you should be able to:

- Explain the role and structure of Cassandra tables.
  - Create, alter, and delete tables.
  - Understand table-level options like TTL and clustering order.
- 

## Table Basics

- **Tables are created *inside* keyspaces** — there's no default keyspace.
- A table stores data using a defined **schema** at the *cluster and node* level.
- Use the syntax:
- ```
CREATE TABLE IF NOT EXISTS keyspace_name.table_name (  
  column1 datatype,  
  column2 datatype,  
  ...  
  PRIMARY KEY (partition_key [, clustering_key])  
);
```

🔑 Primary Keys: Static vs Dynamic

Table Type	Key Structure	Notes
Static	Primary Key = Partition Key only	One row per partition
Dynamic	Primary Key = Partition Key + Clustering Key	Multiple rows per partition

- You can **declare the primary key** either inline or at the end.
- **Static columns** (with `STATIC` keyword) are shared across rows within a partition — like metadata.

🔧 Table Properties and Options

Property	Description
<code>CLUSTERING ORDER BY</code>	Set sort order (default: ascending)
<code>default_time_to_live (TTL)</code>	Sets expiration for <i>all</i> rows in the table
Memtable flush policy	Data is flushed to disk when memtable is full, commit log is full, or at intervals

🕒 TTL can also be set per-row during `INSERT` or `UPDATE`.

🔧 Altering Tables

Supported	Not Supported
Add/drop regular columns	Changing partition key
Rename regular/clustering columns	Changing primary key
Change table properties	Changing column data types

- 🔒 **Primary keys are immutable** — if you want to change them, **create a new table and migrate** the data.
-

Dropping or Clearing Tables

Command	Effect
TRUNCATE	Deletes all data but keeps schema; requires <i>ALL</i> consistency
DROP	Removes both data and schema

 Cassandra takes a **snapshot backup** before `DROP` or `TRUNCATE` — just in case.

Summary of Key Concepts

- Tables are **logical containers** for partitioned data.
 - You can define **ordering**, **TTL**, and other properties.
 - Modify tables with care — **primary key = permanent marriage** 🔒.
 - Use `TRUNCATE` to clear data, or `DROP` to eliminate the whole table.
-

Cassandra CRUD – INSERT & UPDATE Operations (Part 1)

Objectives

By the end of this lesson, you will:

- Understand **how writes happen** in Cassandra.
 - Learn how **INSERT and UPDATE** operations work.
 - Discover when to use **Lightweight Transactions (LWTs)**.
-

How Cassandra Handles Writes

- **Coordinator Node:** The node receiving the write becomes the *orchestra conductor* 🎵.
- **Writes go to all replicas** of a partition (based on replication factor).
- **Consistency Level:** Operation succeeds only if the *minimum number of acknowledgments* is met.

 Example: RF = 3, Consistency = 2 → Write sent to 3 nodes, success if 2 respond.

Node-Level Write Behavior

- **No read-before-write:** Cassandra writes straight to memory → flushed to SSTables on disk.
 - **Write operations are timestamped:** Most recent write wins.
 - **Flush triggers:**
 - Memtable full
 - Commit log full
 - Periodic flush (optional)
-

INSERT Basics

- Requires **full primary key**.
 - Acts as **INSERT or UPSERT**:
 - If the key exists → update
 - If not → insert new record
 - Can omit non-primary key columns.
 - Supports **TTL** at row level:
 - `INSERT INTO table (...) VALUES (...) USING TTL 10;`
-

UPDATE Behavior

- Requires full primary key **except** when updating a **static column** (only partition key needed).
 - **Same as INSERT:** no read-before-write.
 - UPDATE on a non-existing row → inserts it.
-

Lightweight Transactions (LWT)

AKA “If-checks” for INSERT and UPDATE (like optimistic locking 🗨️)

- **Syntax:**
 - `INSERT INTO ... IF NOT EXISTS;`
 - `UPDATE ... IF EXISTS;`
 - `UPDATE ... IF column = value;`
 - Enforces **read-before-write** logic.
 - **4x slower** than standard operations — use *wisely and sparingly*.
-

Summary

Concept	Explanation
Coordinator	Node receiving the write
No read-before-write	Default for performance
INSERT = UPSERT	Inserts new or updates existing
TTL	Expiration time for rows
Static column update	Requires only partition key
LWTs	Safer, but slower operations using <code>IF</code>

Cassandra CRUD Operations – Part 2: READ & DELETE

Objectives

- Understand **how Cassandra handles read operations** across the cluster.
 - Learn **best practices for SELECT queries**.
 - Master **deletion operations** and understand how **tombstones** work.
-

Read Operations

Cluster-Level Mechanics

- Read request hits any node → that node becomes **coordinator**.
- Coordinator sends read to **only the number of replicas** based on `consistency level`.
- **Conflicts resolved by timestamps** (most recent data wins).

SELECT Best Practices

Pattern	Works?	Performance
Start with partition key	✓	★ Best
Use partition key + clustering key(s)	✓	★ Best
Use <code>IN</code> on partition key	✓	⚠ Okay (careful with size)
<code>SELECT *</code> (full scan)	⚠ Works	✗ Very poor
Filter clustering key without partition key	✗	✗ Won't work
Filter regular column	✗	✗ Won't work
Use index on regular column	✓	⚠ Works, but not optimal
Use index + partition key	✓	★ Optimal

 **Golden Rule:** *Always begin your `SELECT` query with the partition key.*

Delete Operations

What Can Be Deleted?

- **Single row** using full primary key ✓
- **Single column** in a row ✓
- **Range of clustering keys** in a partition ✓
- **Entire partition** by partition key ✓

Deletion Warnings

- Use deletes **sparingly** — they create overhead.
- Deleting in Cassandra = writing a **tombstone** (a marker, not instant deletion).
- **Data remains on disk** until `gc_grace_seconds` expires (default: **10 days**).

Tombstones: Cassandra's “Ghost Notes”

- A **tombstone** marks deleted data with a timestamp.
- Even though data isn't visible, it's **still on disk** until garbage collection.
- Helps maintain **consistency across nodes**.
- Too many tombstones =  performance nightmares.

Example: What happens when one replica missed the delete?

- Node 1 and 3 receive delete at T1 → write tombstone.
- Node 2 still has the original data (T0).
- On read: Node 1 returns tombstone (T1), Node 2 returns value (T0).
- Coordinator sees T1 is newer → returns **no data**.

Summary Table

Feature	Behavior
Read coordinator	Node that receives the read
SELECT performance	Best when ordered by PK & starts with partition key
Index on regular column	Works, but better used with partition key
Deleting data	Creates tombstones
Tombstone TTL	<code>gc_grace_seconds</code> (default 864000 = 10 days)

🚫 *Tip of the day:* "Don't delete like you're angry at the data. Cassandra forgives, but its tombstones don't forget... immediately." 😊

📖 Glossary: Working with Cassandra

Term	Definition
Cluster	Group of interconnected nodes that store/manage data with high availability and fault tolerance.
Consistency	Ensures all nodes in the distributed system return the same data at the same time.
Dynamic Table	A table with flexible schema—columns can vary by row.
Keyspace	Top-level organizational unit in Cassandra, similar to a database in RDBMS.
Lightweight Transactions	Resource-intensive operations that provide stronger consistency using conditions like <code>IF NOT EXISTS</code> .
Partition Key	Component of the primary key that determines how data is distributed across the cluster.
Primary Key	Uniquely identifies rows in a table; includes partition key and optional clustering columns.
Replication Factor	Number of data copies maintained across different nodes for redundancy.
Replicas	Actual copies of the data stored on multiple nodes.
Replication Strategy	Rules for distributing replicas across nodes and data centers.
Secondary Indexes	Allow querying based on non-primary key columns.
Static Table	Table with a fixed set of columns per row (rigid schema).
Table	Structure for storing related data, organized into rows and columns within a keyspace.

Summary and Highlights

- Cassandra supports built-in, collection, and user-defined data types.
- Both collection and user-defined data types offer a way to group and store data.
- Keyspaces are defined before creating tables, and a keyspace can contain any number of tables.

- Common keyspace operations are CREATE KEYSPACE, ALTER KEYSPACE, and DROP KEYSPACE.
- A table's metadata specifies the primary key and instructs Cassandra on distributing the table data at the cluster and node level.
- Cluster-level writes are sent to all the partition's replicas, irrespective of the consistency factor.
- By default, Cassandra doesn't perform a read before writes; therefore, INSERT and UPDATE operations behave similarly.
- Reads at cluster level are sent only to the number of replicas according to the consistency setting.
- Reads follow the primary key column's order for best performance.

Glossary: Introduction to NoSQL (A–F)

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability – guarantees for RDBMS transactions.
Aggregation	Summarizing and computing data values.
Aggregation pipeline	MongoDB's framework for data transformation using staged operations.
Atomic	Ensures a database operation is indivisible; either fully completes or is rolled back.
Availability	System remains operational and responsive despite failures.
B+ Tree	A data structure for efficient indexing in databases.
BASE	Basically Available, Soft state, Eventually consistent – alternative to ACID.
Basically available	System stays operational during partial failures.
Bigtable	Google's scalable NoSQL database for large datasets.
BSON	Binary format similar to JSON, used for efficient MongoDB storage.
Caching	Temporarily storing frequently accessed data for faster retrieval.
CAP	Consistency, Availability, Partition tolerance – tradeoff model in distributed systems.
Cluster	Group of nodes working together to manage NoSQL data with high availability.
Clustering key	Defines the sort order of rows within a partition in Cassandra.
Code-first	Development style where app code defines the schema.
Collection	MongoDB's grouping of related JSON-like documents.

Term	Definition
Column database	NoSQL model storing data in columns instead of rows (e.g., Cassandra).
Consistency	Ensures all nodes reflect the same data at a given time.
Consistent	A property ensuring valid state transitions during transactions.
CQL	Cassandra Query Language – SQL-like interface for Cassandra.
CQL shell	Command-line tool to run CQL commands.
CRUD	Create, Read, Update, Delete – basic data operations.
Database	Logical container for collections in MongoDB.
DBaaS	Database as a Service – managed database hosting in the cloud.
Decentralized	No central controller – all nodes equal in responsibility.
Denormalized	Design choice to duplicate data for faster queries (common in NoSQL).
Document	Semi-structured data format (e.g., JSON/BSON) in document databases.
Durable	Ensures committed transactions survive system failures.
Dynamic table	Table where columns can vary across rows.
Election	MongoDB's process to select a new primary node during failover.
Eventually consistent	Data becomes consistent over time in distributed systems.
Expressive querying	Ability to write flexible, powerful queries (notably in MongoDB).
ETL	Extract, Transform, Load – data migration/integration process.

Glossary: Introduction to NoSQL (G–L)

Term	Definition
Graph database	NoSQL model optimized for storing complex relationships using nodes and edges.
High availability (HA)	Ensures database remains operational with minimal downtime.
Horizontal scaling	Adding nodes to handle increased data/traffic.
Idempotent changes	Operations that can be repeated without side effects.
Indexing	Creating data structures to speed up query performance.
Isolated	Ensures concurrent transactions don't interfere.
Joins	Combining data from multiple sources based on common attributes.
JSON	Human-readable data-interchange format used in NoSQL databases.
Keyspace	Top-level container in Cassandra (like a DB in RDBMS).
Key-value	Simple NoSQL model storing key-value pairs.

Term	Definition
Lightweight transactions	Conditional operations with strong consistency, slower than normal ops.

Glossary: Introduction to NoSQL (M–R)

Term	Definition
mongo shell	MongoDB CLI for executing database commands interactively.
MongoClient	API client to connect apps with MongoDB.
MQL	MongoDB Query Language for querying data.
Normalized	Organized schema design to reduce redundancy.
NoSQL	“Not Only SQL” – flexible databases beyond traditional tables.
Operational data	Actively used application data.
Oplog	MongoDB log of operations for replication.
Partition key	Used to distribute data across nodes in Cassandra.
Partition tolerance	System resilience during network partitioning.
Partitions	Unit of data storage in Cassandra based on partition keys.
Peer-to-peer	Equal-node architecture with no central master (e.g., Cassandra).
Primary node	Main writable node in a MongoDB replica set.
Primary key	Unique identifier for a record, includes partition/clustering keys.
Replicas	Data copies distributed across nodes for availability.
Replication	Process of copying data to ensure availability and fault tolerance.
Replication factor	Number of replicas to maintain.
Replication lag	Delay between primary and secondary data sync.
Replication strategy	Rules for placing replicas across nodes/data centers.

Glossary: Introduction to NoSQL (S–Z)

Term	Definition
Scalability	System's ability to handle growing data/traffic loads.
Secondary	Read-only replica nodes in MongoDB.
Secondary indexes	Enable querying non-primary key columns.
Sharding	Splitting a database into chunks (shards) for distribution.
Soft state	Temporary inconsistencies are acceptable and resolved over time.
Static table	Table with a fixed set of columns.

Term	Definition
Table	Organized data in rows and columns (RDBMS or NoSQL context).
Transactions	Group of operations treated as a single unit.
TTL (Time to Live)	Expiration mechanism for auto-deleting data after a time period.
Unstructured data	Schema-less, flexible data format like JSON.
Vertical scaling	Upgrading hardware to increase performance.
XML	Extensible Markup Language – alternative to JSON for structured data.

IBM Cloudant – Summary of Key Concepts

Overview of IBM Cloudant

- **Type:** Fully managed distributed NoSQL document-oriented **Database-as-a-Service (DBaaS)**.
 - **Based on:** Open-source **Apache CouchDB** with HTTP API and JSON document storage.
 - **Ideal for:** Web, mobile, IoT, and serverless applications.
 - **Highlights:**
 - Schema flexibility.
 - Online analytics, full-text search (Apache Lucene), geospatial querying (GeoJSON).
 - **Offline-first support** with **Cloudant Sync**.
 - 24/7 support and automated maintenance.
-

Architecture & Technologies

- **Deployment:** 55+ global data centers, cloud-agnostic (IBM, AWS, Azure, RackSpace).
 - **Durability & HA:** Automatic geographic replication, zone redundancy.
 - **Security:** ISO27001, SOC 2 Type 2, HIPAA-ready, encryption (at rest & transit), key management (IBM Key Protect).
 - **Core techs:**
 - RESTful HTTP API (JSON format).
 - Built-in **MapReduce**, **search**, and **geospatial features**.
 - Integrated dashboard for management.
-

Deployment Plans

Plan	Purpose	Limits/Notes
Lite Plan	Free forever for development & prototyping	Limited throughput (20 reads/sec, 10 writes/sec), 1MB doc limit
Standard Plan	Serverless scaling for production	Pay-per-hour based on provisioned capacity (not usage)
Dedicated Hardware Plan	Secure, isolated environment	Single-tenant, HIPAA-compliant optional, monthly fixed cost

Dashboard Features

- **Sections:**
 - Databases (create, delete, manage docs).
 - Replication (configure one-time or continuous sync).
 - Active Tasks (monitor indexing, replication).
 - Monitoring (reads/writes, denied requests, storage usage).
 - **Replication Notes:**
 - Continuous replication adds cost & system load.
 - Replications can impact performance—test accordingly.
-

Databases & Documents

- **Types:**
 - **Partitioned:** Offers **partitioned + global queries**; best performance.
 - **Non-Partitioned:** Supports only global queries; legacy format.
 - **Documents:**
 - Must be valid JSON, $\leq 1\text{MB}$, uniquely identified by `_id`.
 - CRUD operations via UI or API.
 - **Views:** Metadata, Table, and JSON format.
-

HTTP API & `curl`

- **API Access:** Via standard HTTP (no drivers needed).
 - **Tooling:** Uses `curl`, aliases (`acurl`), and variables for ease.
 - **Supported methods:**
 - GET – Read data.
 - PUT – Create/Update database/doc.
 - POST – Run queries, create documents/indexes.
 - DELETE – Drop databases/docs (requires revision token).
 - **Querying:**
 - MongoDB-style **JSON selectors**.
 - POST to `/_find` endpoint.
 - Supports logic operators (`$gt`, `$lt`, etc.).
 - Store complex queries in `.json` files and reference via `-d@file.json`.
-

Benefits & Use Cases

- **Benefits:**
 - Extreme scalability (from GBs to PBs).
 - Always-on availability.
 - Data durability (triple-replication).
 - Online upgrades with zero downtime.
 - Simplified infrastructure (no admin needed).
 - **Use Cases:**
 - Point-of-sale mobile apps.
 - IoT data analysis.
 - Cognitive AI apps (e.g. IBM Watson integrations).
-